

Quantum Complexity Theory

John Watrous
Institute for Quantum Computing
University of Waterloo

Part I

Classical complexity primer

1	Promise problems and Turing machines	3
1.1	Strings, languages, and promise problems	3
1.2	One-tape Turing machines	9
1.3	Multi-tape Turing machines	16
1.4	Running time and space	20
2	Classical complexity classes	23
2.1	Deterministic time and space	23
2.2	Complexity classes via quantifiers	30
2.3	Relationships and the containment diagram	38
3	Reductions, completeness, and Boolean circuits	41
3.1	Polynomial-time and log-space computable functions	41
3.2	Reductions and completeness	43
3.3	Boolean circuits	47
3.4	The Cook–Levin theorem	52

Chapter 1

Promise problems and Turing machines

Quantum complexity theory builds on classical complexity theory. It is therefore appropriate to begin the course with an overview of classical complexity theory, which will provide us with a strong foundation that will serve us well for the remainder of the course. In this first chapter, we will discuss some basic definitions from theoretical computer science, including the Turing machine model of computation.

1.1 Strings, languages, and promise problems

Alphabets

An *alphabet* is a set of symbols that can be used for writing, storing information, and performing computations. For example, the *binary alphabet* is the set $\{0, 1\}$ containing just the symbols 0 and 1, while the upper-case Roman alphabet is the set $\{A, B, C, \dots, Z\}$ containing the 26 upper-case Roman letters.

When we speak of a *symbol* in this context, the intended meaning is a recognizable character, mark, or glyph of some sort. The symbols we encounter in everyday life are often rooted in history or are selected in intuitive ways that aid in their recognition, but from a mathematical viewpoint the details of the symbols themselves are unimportant. What is important is that symbols are unambiguously recognizable and distinguishable from one another — but that is all. Correspondingly, the technical definition of an alphabet makes no attempt to tease out what exactly

makes a symbol a symbol, and instead captures just the mathematical essence of the concept.

Definition 1.1. An *alphabet* is a finite and nonempty set whose elements are called *symbols*.

In particular, the notion of a set already captures the idea of two or more objects being distinct, and beyond this the only additional constraints on the symbols in an alphabet are that there is at least one and there are only finitely many different ones. The requirement that alphabets are finite is not only standard but is also an eminently reasonable reflection of the notion of a symbol — for we must presumably have seen a symbol before within our finite lifetimes if it is to be recognized and carry meaning.

For example, consider the set

$$\mathbb{Z}_n = \{0, \dots, n-1\} \quad (1.1)$$

for any positive natural number $n \in \mathbb{N}^\times = \{1, 2, 3, \dots\}$. Such a set is finite and nonempty, and is therefore an alphabet. In particular, $\mathbb{Z}_2$ is the binary alphabet. To be clear, the elements of $\mathbb{Z}_n$ are being viewed abstractly as n different, unambiguously distinguishable symbols in this context.

To some, it might seem questionable to consider a set such as $\mathbb{Z}_{1000000}$, which has one million different elements, to be an alphabet and to view its elements as symbols. However, despite practical concerns about the difficulty of remembering and recognizing one million different symbols, there is no fundamental barrier to contemplating such an alphabet. Of course, practically speaking, alphabets are usually much smaller than this. For instance, we conventionally use the *decimal alphabet* $\mathbb{Z}_{10} = \{0, 1, \dots, 9\}$ to express numbers using decimal notation, and in doing this we represent each element of $\mathbb{Z}_{1000000}$ as a sequence of up to six numerals (such as 978223, for instance), rather than by a single symbol.

Alphabets can contain any symbols a person might dream up — like the following alphabet, whose symbols represent chess pieces.

$$\{\text{♔}, \text{♚}, \text{♖}, \text{♗}, \text{♘}, \text{♙}\} \quad (1.2)$$

Using this alphabet in a context involving the game of chess could be quite practical. Mathematically speaking, however, these symbols carry no meaning by themselves, and there is no fundamental distinction between this alphabet and $\mathbb{Z}_6$.

Indeed, when one wishes to establish mathematical facts involving an alphabet, it is sometimes convenient to assume that the alphabet in question is $\mathbb{Z}_n$ for a

suitable positive natural number n , and no loss of generality is incurred in doing this. That is, any n arbitrary symbols that form an alphabet could equally well be exchanged for the symbols $0, \dots, n-1$ without changing their essential nature (i.e., that these are n identifiable and distinguishable symbols) or any relevant mathematical facts concerning them as symbols.

On the other hand, it can also be convenient in some circumstances to choose to use symbols other than $0, 1, 2$, and so on, particularly when conventions are in place that suggest special roles for different symbols. Using symbols based on chess pieces in a context involving the game of chess is a fine example. A more common example in theoretical computer science is that the symbol $\#$ is often used as a clearly recognizable separator in a string, such as in $110\#0100$, where two binary strings 110 and 0100 have been combined into a single string by including the separator symbol $\#$ between them. While it is true that the symbol 2 could be used in place of $\#$ to serve the same purpose, the symbol $\#$ is more immediately recognizable as it is.

In general, it is typical that alphabets are named with capital Greek letters such as Σ and Γ .

Strings

A *string* is a finite sequence of symbols, as the following definition states.

Definition 1.2. A string is a finite sequence of symbols from an alphabet. This includes the possibility of an empty sequence (i.e., one containing no symbols at all).

It is also perfectly reasonable to consider *infinitely long* sequences of symbols — such as the digits in a decimal expansion of π for instance — but as a matter of terminology such sequences will not be called strings in this course. That is to say, strings are finite *by definition*.

Finite sequences are generally written inside of parentheses using commas to separate the elements, like for the sequence $(1, 2, 5, 3)$. For strings, however, the standard convention is to write them by simply juxtaposing the symbols in order, rather than using parentheses and commas. For instance, as a string of symbols drawn from the decimal alphabet $\mathbb{Z}_{10}$, the sequence $(1, 2, 5, 3)$ is written 1253 . Mathematically speaking, it is the same sequence as $(1, 2, 5, 3)$, but the juxtaposed form is more concise and more immediately connected with the intuitive notion of a string.

Often we wish to indicate explicitly the alphabet from which the symbols in a string are drawn, and the standard phrasing for doing this is to refer to a string *over* a particular alphabet. For example, these are strings over the binary alphabet:

$$1101001, \quad 110, \quad \text{and} \quad 00. \quad (1.3)$$

By the way, it is correct to say that 00 is a string over the binary alphabet, despite the fact that it only includes the symbol 0; it just does not happen to include the symbol 1. It is also correct to say, however, that 00 is a string over the *unary alphabet* $\mathbb{Z}_1 = \{0\}$. In general, a string is a string over a particular alphabet if all of its symbols are elements of that alphabet, whether or not every symbol in the alphabet actually appears in the string.

Strings are typically named using lower-case Roman letters near the end of the alphabet such as u, v, w, x , and so on. For example, one might write

$$x = 1101001, \quad (1.4)$$

in which case x is a string over the binary alphabet.

String length

The *length* of a string is its length as a sequence — meaning the total number of symbols it contains, including repetitions. For instance, the length of the string 1101001 is 7 and the length of the string 00 is 2. The length of a string is denoted by placing vertical bars on both sides of the string, just like the standard notation for the absolute value of a number. For example,

$$|1101001| = 7, \quad (1.5)$$

and if $x = 1101001$ we may instead write $|x| = 7$.

The empty string

The string containing no symbols at all is called the *empty string*. Despite its apparent uselessness, this is an important string, essentially playing the role of 0 for strings rather than numbers.

Obviously, it would be hopelessly confusing to refer to the empty string in technical writing by literally writing no symbols at all — so we choose the special name ε to refer to the empty string. Having made this choice, we will vow never to select the Greek letter ε as a symbol in an alphabet, to avoid the confusion this

would cause. (We will, however, use ε to refer to small, positive real numbers — but this will be clear from context and will not cause any confusion.) The letter λ is a common alternative to ε for representing the empty string.

The empty string has length 0, which we may express symbolically as $|\varepsilon| = 0$, and it is the unique string having this property. Note that the empty string may be regarded as a string over any chosen alphabet Σ — it contains no symbols at all, so vacuously all of its symbols are drawn from whatever alphabet Σ we might wish to consider.

Additional notation for certain sets of strings

The set of all strings over an alphabet Σ is denoted Σ^* . For example,

$$\{0,1\}^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, 000, 001, \dots\}. \quad (1.6)$$

The specific pattern $\varepsilon, 0, 1, 00, 01, 10, 11, 000, 001, \dots$ suggested in (1.6) is commonly known as the *lexicographic* ordering of the binary strings.¹ In words, strings are ordered by length, together with “dictionary” ordering among strings having the same length, defined by ordering the alphabet symbols with 0 before 1.

The lexicographic ordering can be generalized to an arbitrary alphabet, provided that the symbols themselves are given an order, thereby defining what is meant by “dictionary ordering” among strings having the same length. For example, the lexicographic ordering of Σ^* for $\Sigma = \{A, B, \dots, Z\}$ is commonly used to label columns in spreadsheets.

$$\varepsilon, A, B, \dots, Z, AA, \dots, AZ, BA, \dots, BZ, \dots, ZA, \dots, ZZ, AAA, \dots \quad (1.7)$$

For a given alphabet Σ and a natural number $n \in \mathbb{N}$, the notation Σ^n refers to the set of all strings over the alphabet Σ having length equal to n , while $\Sigma^{\leq n}$ refers to the set of all strings over Σ having length at most n . For example, we have the following sets of strings.

$$\begin{aligned} \{0,1\}^3 &= \{000, 001, 010, 011, 100, 101, 110, 111\} \\ \{0,1\}^{\leq 2} &= \{\varepsilon, 0, 1, 00, 01, 10, 11\} \end{aligned} \quad (1.8)$$

For every alphabet Σ , it is the case that $\Sigma^0 = \{\varepsilon\}$, as the empty string is the only string of length 0.

¹This is a term for which there is not universal agreement. Some instead use the name *quasi-lexicographic ordering* for this pattern and mean something slightly different by the term *lexicographic ordering*.

Languages

The term *language* refers to any set of strings over some alphabet, as the following definition makes clear.

Definition 1.3. A *language* over an alphabet Σ is any set of strings over the alphabet Σ .

Notice that there must be a single alphabet Σ over which all of the strings in the language are defined. We would not, for instance, consider a set of strings that includes infinitely many different symbols appearing among all of the strings to be a language.

Traditionally speaking, languages have been associated with *decision problems* in complexity theory, where a string is given as input and the goal is to determine whether that string is or is not an element of a particular language. Promise problems, which will be discussed shortly, extend the notion of decision problems in a useful and practical way by effectively restricting the set of valid inputs and treating other strings as “don’t care” inputs.

A simple example of a language over a given alphabet Σ is Σ^* , the set of all strings over Σ . Another example is the *empty language*, the set containing no strings at all, which is a language over an arbitrary choice of an alphabet. The empty language is denoted $\emptyset$, which is the standard notation for the empty set.

Here are a few less trivial examples of languages, all of which are languages over the binary alphabet.

$$A = \{0010, 110110, 011000010110, 111110000110100010010\} \quad (1.9)$$

$$B = \{x \in \{0,1\}^* : x \text{ starts with } 0 \text{ and ends with } 1\} \quad (1.10)$$

$$C = \{x \in \{0,1\}^* : x \text{ is a binary representation of a prime number}\} \quad (1.11)$$

$$D = \{x \in \{0,1\}^* : |x| \text{ and } |x| + 2 \text{ are prime numbers}\} \quad (1.12)$$

The language A is a *finite* language, meaning that it contains finitely many different strings (four in this case). The languages B and C , on the other hand, are not finite — they both include infinitely many different strings. At the time of the writing of these notes, it is not known if D is infinite, for determining whether or not this is the case is equivalent to resolving the *twin primes conjecture*.

Promise problems

The notion of a *promise problem* is especially important in quantum complexity. Promise problems are decision problems, meaning that the goal is to discriminate

between two possible alternatives: *yes-inputs* and *no-inputs* of the problem. The word *promise* refers to a guarantee that the input possesses a particular property.

Formally speaking, a promise problem is a pair

$$A = (A_{\text{yes}}, A_{\text{no}}), \quad (1.13)$$

where $A_{\text{yes}}, A_{\text{no}} \subseteq \Sigma^*$ are sets of strings over some alphabet Σ satisfying

$$A_{\text{yes}} \cap A_{\text{no}} = \emptyset. \quad (1.14)$$

The strings in the sets A_{yes} and A_{no} are called the *yes-inputs* and *no-inputs* of the problem, respectively. The correct answer to a promise problem $(A_{\text{yes}}, A_{\text{no}})$ on an input $x \in A_{\text{yes}}$ is yes (or, alternatively, 1 or *accept*), while the correct answer on an input $x \in A_{\text{no}}$ is no (or, alternatively, 0 or *reject*). Inputs in $\Sigma^* \setminus (A_{\text{yes}} \cup A_{\text{no}})$ are treated as “don’t care” inputs, and no requirements are placed on computations on such inputs.

The decision problem associated with a given *language* treats the strings in that language as yes-inputs and the strings not in that language as no-inputs of the problem. Such problems can be expressed as promise problems for which the promise is trivial — so every input string over the alphabet of the language is considered to be a valid input. More precisely, we may take A_{yes} to be the language in question while $A_{\text{no}} = \Sigma^* \setminus A_{\text{yes}}$, which implies that

$$A_{\text{yes}} \cup A_{\text{no}} = \Sigma^*. \quad (1.15)$$

For promise problems in general, this equality need not hold.

Although complexity theory has traditionally focused on languages rather than promise problems, much is gained in shifting one’s focus to promise problems. In particular, promise problems allow for the formalization of decision problems having specific domains of interest that often arise naturally in computational settings. This shift is especially significant for the material we will study in this course: the central quantum complexity classes we will consider (including BQP and QMA, among others) are most naturally defined as classes of promise problems. Correspondingly, we will treat promise problems (rather than languages) as the default framing for decision problems.

1.2 One-tape Turing machines

The Turing machine model is a cornerstone of complexity theory. There are alternative computational models that could equally well serve in its place, but its

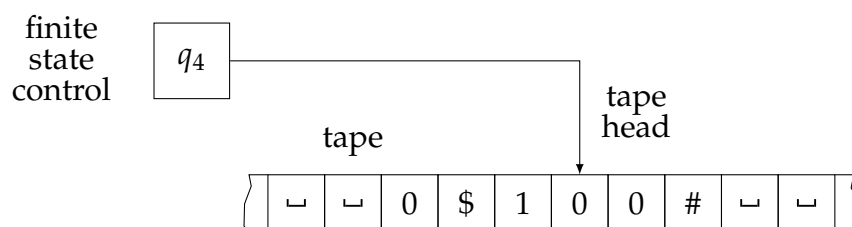


Figure 1.1: An illustration of the three components of a Turing machine: the finite state control, the tape head, and the tape. The tape is infinite in both directions (although it appears that this Turing machine's tape was torn at both ends to fit it into the figure).

simplicity and historical significance have cemented its status as a default computational model in complexity theory. We will begin with one-tape Turing machines, which are the simplest variant of the model, and then move on to a more versatile multi-tape variant in the next section.

There are three components of a one-tape Turing machine:

1. The *finite state control*. This component is capable of storing one of a finite number of different *states* at each instant.
2. The *tape*. This component consists of an infinite number of *tape squares*, each of which can store one of a finite number of *tape symbols* at each instant. The tape is infinite both to the left and to the right.
3. The *tape head*. The tape head can move left and right on the tape, and is understood to be scanning exactly one of the tape squares at the start of each computational step. The tape head can read which symbol is stored in the square it scans, and it can write a new symbol into that square.

Figure 1.1 illustrates these three components. It is natural to imagine that the tape head is connected in some way to the finite state control.

The action of a Turing machine at each instant is determined by the state of the finite state control together with the symbol stored in the tape square that the tape head is currently reading. Thus, the action is determined by a finite number of possible alternatives: one action for each state/symbol pair. Depending on the state and the symbol being scanned, the action that the machine is to perform may involve changing the state of the finite state control, changing the symbol in the tape square being scanned, and moving the tape head to the left or right. Once

this action is performed, the machine will again have some state for its finite state control and will be reading some symbol on its tape, and the process continues.

When a Turing machine begins a computation, an input string is written on its tape in consecutive squares, one symbol per square. Every other tape square is initialized to a special *blank* symbol, which may not be included in the alphabet over which input strings are assumed to be drawn. In these notes, we will use the symbol $\sqcup$ for this purpose.

We also give Turing machines an opportunity to stop the computational process and produce a final decision: *accept* or *reject*. Two special states of the finite state control are reserved for this purpose: an *accept* state q_{acc} and a *reject* state q_{rej} . These two states are deemed *halting states*, and all other states are *non-halting states*. If the machine enters a halting state, the computation immediately stops and *accepts* or *rejects* accordingly.

For a given decision problem, our focus is on whether or not a given Turing machine eventually reaches one of the states q_{acc} or q_{rej} on a given input string, thereby stopping the computation and accepting or rejecting that string. One can also use the Turing machine model to describe the computation of *functions*, where an output string is produced, by taking into account the contents of the tape if and when a halting state is reached.

Formal definition

With the informal description of Turing machines from above in mind, we will now proceed to the formal definition.

Definition 1.4. A *one-tape deterministic Turing machine* (or 1-DTM, for short) is a 7-tuple

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{acc}}, q_{\text{rej}}), \quad (1.16)$$

where Q is a finite and nonempty set of *states*, Σ is an alphabet called the *input alphabet*, which may not include the blank symbol $\sqcup$, Γ is an alphabet called the *tape alphabet*, which must satisfy $\Sigma \cup \{\sqcup\} \subseteq \Gamma$, δ is a transition function having the form

$$\delta : (Q \setminus \{q_{\text{acc}}, q_{\text{rej}}\}) \times \Gamma \rightarrow Q \times \Gamma \times \{\leftarrow, \rightarrow\}, \quad (1.17)$$

$q_0 \in Q$ is the *initial state*, and $q_{\text{acc}}, q_{\text{rej}} \in Q$ are the *accept* and *reject* states, which must satisfy $q_{\text{acc}} \neq q_{\text{rej}}$.

The interpretation of the transition function is as follows. Suppose the 1-DTM is currently in a state $p \in Q$, the symbol stored in the tape square being scanned

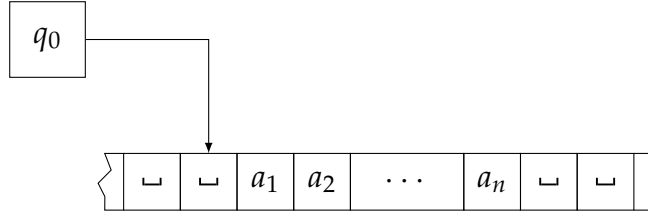


Figure 1.2: The initial configuration of a 1-DTM when run on input $w = a_1 a_2 \cdots a_n$.

by the tape head is $a \in \Gamma$, and it is the case that

$$\delta(p, a) = (q, b, D) \quad (1.18)$$

for $D \in \{\leftarrow, \rightarrow\}$. The action performed by the 1-DTM is then to

1. change state to q ,
2. overwrite the contents of the tape square being scanned by the tape head with b , and
3. move the tape head in direction D (either left or right).

In the case that the state is q_{acc} or q_{rej} , the transition function does not specify an action, because we assume that the 1-DTM halts once it reaches one of these two states.

Turing machine computations

If we have a 1-DTM $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{acc}}, q_{\text{rej}})$, and we wish to consider its operation on some input string $w \in \Sigma^*$, we assume that it is started with its components initialized as illustrated in Figure 1.2. That is, the input string is written on the tape, one symbol per square, with every other tape square containing the blank symbol, and with the tape head scanning the tape square immediately to the left of the first input symbol. In the case that the input string is ε , all of the tape squares start out storing blanks.

Once the initial arrangement of the 1-DTM is set up, the 1-DTM begins taking *steps*, as determined by the transition function δ in the manner suggested above. So long as the 1-DTM does not enter one of the two states q_{acc} or q_{rej} , the computation continues. If the 1-DTM eventually enters the state q_{acc} , it *accepts* the input string, and if it eventually enters the state q_{rej} , it *rejects* the input string. Thus, there are three possible alternatives for a 1-DTM M on a given input string w :

1. M accepts w .
2. M rejects w .
3. M runs forever on input w .

In some cases one can design a particular 1-DTM so that the third alternative does not occur, but in general it might.

There is a rigorous formalization of Turing machine computations based on the description above, the details of which can safely be skipped for the purposes of this course. In summary, one first defines the notion of a *configuration* of a Turing machine, which records the state of the finite control, the contents of the tape, and the position of the tape head at a given instant (i.e., essentially everything there is to say about a given Turing machine at a given moment). The transition function then defines a so-called *yields relation* on the set of all configurations, indicating precisely when one configuration leads to another after one computational step.

For example, the configuration suggested by Figure 1.1 can be expressed symbolically as

$$0\$1(q_4, 0)0\#, \quad (1.19)$$

where the non-blank symbols $0\$1$ appear on the tape to the left of the tape head, the symbol 0 appears under the tape-head, the state is q_4 , and the non-blank symbols $0\#$ appear to the right of the tape head. Supposing that the transition function δ satisfies

$$\delta(q_4, 0) = (q_7, 1, \rightarrow), \quad (1.20)$$

meaning that when reading the symbol 0 when in state q_4 the 1-DTM changes state to q_7 , writes 1 on the scanned tape square, and moves right, the 1-DTM would then enter the configuration

$$0\$11(q_7, 0)\# \quad (1.21)$$

as a result. We then write

$$0\$1(q_4, 0)0\# \vdash_M 0\$11(q_7, 0)\# \quad (1.22)$$

to indicate this relationship.

Finally, a computation is a sequence of configurations, each yielding the next, starting from the initial configuration on a chosen input string. While intuitively straightforward, there are some technical details required to express these definitions in a precise way. We will not do this here, but it should be understood that whenever we refer informally to what a Turing machine does on a given input, there is a fully rigorous mathematical meaning to such statements.

An example: a 1-DTM deciding $\{0^n 1^n : n \in \mathbb{N}\}$

Here is a specific example of a 1-DTM that can safely be skipped — its purpose is only to exhibit a small 1-DTM in complete detail through a so-called *state diagram* (also called a *state transition diagram*).

A *state diagram* for a 1-DTM is a directed graph whose nodes are the states of the machine and whose edges are labelled in one of the following forms.



The form on the left indicates transition function values of the form

$$\delta(p, a) = (q, b, \leftarrow) \quad (1.23)$$

drawn as an edge from node p to node q . The interpretation of an edge like this is “if the machine is in state p while scanning the symbol a , then overwrite a with b , move the tape head left, and enter the state q .” The form on the right is similar for transition function values of the form

$$\delta(p, a) = (q, b, \rightarrow) \quad (1.24)$$

for which the tape head moves right rather than left. An unattached arrow entering a state identifies it as the initial state q_0 , and a state drawn with a doubled outline is the accept state q_{acc} .

Figure 1.3 gives a state diagram of a 1-DTM M that decides the language

$$\{0^n 1^n : n \in \mathbb{N}\}, \quad (1.25)$$

meaning that it accepts binary strings of the form $0^n 1^n$ (such as ε , 01, 0011, 000111, and so on) and rejects every other binary string.

The machine works as follows. Its tape head begins on the blank immediately to the left of the input. On its first step it moves right and enters state q_1 . If the first non-blank symbol is a 1, the input cannot have the form $0^n 1^n$ for any $n \geq 1$, and M rejects; if it is a blank, the input is empty (the $n = 0$ case) and M accepts; otherwise it is a 0, which M effectively erases (by overwriting the symbol with a blank symbol), moving into state q_2 and scanning rightward across the remainder of the input. Upon reaching the first blank past the end, M backs up one square in state q_3 , expecting a 1: a 0 or a blank at that location means the input was not of the required form, and M rejects. If a 1 is found, M erases it and returns leftward to the

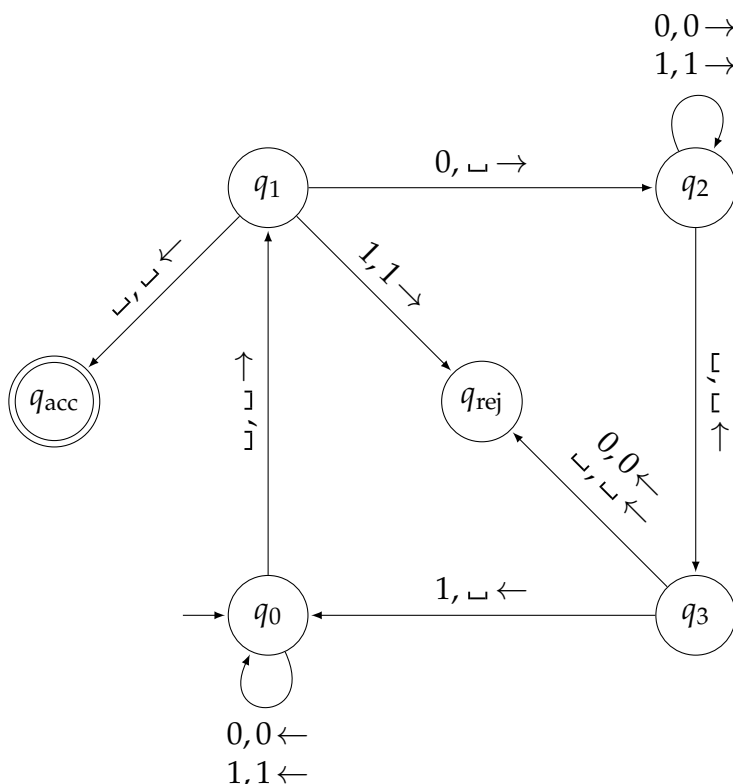


Figure 1.3: A 1-DTM deciding the language $\{0^n 1^n : n \in \mathbb{N}\}$.

blank preceding the tape's non-blank contents. At that point the tape effectively contains a copy of the input with one 0 removed from the start and one 1 removed from the end, and the process begins again. Acceptance occurs when the tape has been emptied.

State diagrams like this one make the transitions of a small 1-DTM completely explicit. However, for all but the simplest machines they become quite unwieldy: a 1-DTM implementing an interesting algorithm would require a diagram with far too many states to fit on one page. More importantly, as a human it is nearly impossible to make sense of such a diagram, not unlike making sense of a complicated program written in assembly language. For this reason, 1-DTMs are more typically described at a higher level, leaving the lower-level details unspecified. In essence, they are described at the level of algorithm pseudo-code or step-by-step procedures involving tapes and symbols rather than state transitions.

For example, the example Turing machine above could alternatively be described as follows:

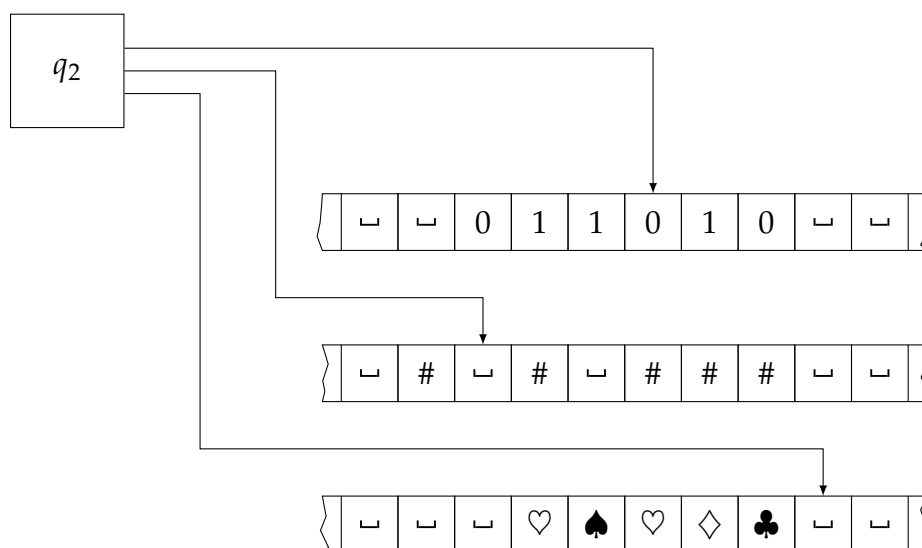


Figure 1.4: A three-tape DTM.

1. Accept if the input is the empty string.
2. Check that the left-most non-blank symbol on the tape is a 0 and that the rightmost non-blank symbol is a 1. Reject if this is not the case, and otherwise erase these symbols and go to step 1.

There are, in fact, several specific ways to implement this algorithm with a 1-DTM, with the 1-DTM described formally in Figure 1.3 being one of them.

1.3 Multi-tape Turing machines

Next we extend the Turing machine model: a *multi-tape deterministic Turing machine* is similar to a one-tape deterministic Turing machine, except that it has k tape heads that operate independently on k tapes, for some fixed positive integer k . In general, we will write DTM as an abbreviation for a multi-tape deterministic Turing machine. (The abbreviation k -DTM is also common, but as multi-tape Turing machines will be our default model, the abbreviation DTM will suffice for our needs.) Figure 1.4 illustrates a DTM with three tapes.

It is convenient to extend the Turing machine model in this way for a few reasons. One is that it is often easier to describe and/or define Turing machines for specific tasks when they have multiple tapes with freely operating tape heads, each of which may be used for a specific purpose (not unlike variables in a computer

program, for instance). Another reason is that multi-tape Turing machines allow us to define sublinear space bounds — such as logarithmic or poly-logarithmic space — in a simple and natural way. In particular, the input string is assumed to be accessed from a dedicated read-only input tape, which cannot be used to store information and therefore does not contribute to space usage. It is also common to consider Turing machines with a dedicated write-only output tape when we wish to model the computation of functions.

In general, a k -tape DTM is defined in a similar way to a 1-DTM, except that the transition function has a slightly more complicated form. In particular, if the tape alphabets of a k -tape DTM are $\Gamma_1, \dots, \Gamma_k$, then the transition function might take this form:

$$\delta : (Q \setminus \{q_{\text{acc}}, q_{\text{rej}}\}) \times \Gamma_1 \times \dots \times \Gamma_k \rightarrow Q \times \Gamma_1 \times \dots \times \Gamma_k \times \{\leftarrow, \rightarrow\}^k. \quad (1.26)$$

If we make the simplifying assumption that the same alphabet is used for each tape (which does not restrict the model, as we could always take this single tape alphabet to be the union $\Gamma = \Gamma_1 \cup \dots \cup \Gamma_k$ of multiple alphabets), the transition function takes the form

$$\delta : (Q \setminus \{q_{\text{acc}}, q_{\text{rej}}\}) \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{\leftarrow, \rightarrow\}^k. \quad (1.27)$$

The interpretation of the transition function taking the form (1.27) is as follows. If it is the case that

$$\delta(p, a_1, \dots, a_k) = (q, b_1, \dots, b_k, D_1, \dots, D_k), \quad (1.28)$$

then if the DTM is in the state p and is reading the symbols $a_1, \dots, a_k$ on its k tapes, then

1. the new state becomes q ,
2. the symbols $b_1, \dots, b_k$ are written onto the k tapes (overwriting the symbols $a_1, \dots, a_k$), and
3. the j -th tape head moves in the direction depending on $D_j \in \{\leftarrow, \rightarrow\}$, for each $j = 1, \dots, k$.

Let us note that the option for any of the tape heads to remain stationary rather than necessarily moving left or right is also common but not important for the sake of this discussion.

It is possible to simulate a (multi-tape) DTM with a 1-DTM, though each step of the DTM requires many steps for the 1-DTM running the simulation. One way to do this, which is suggested informally by Figure 1.5, is as follows.

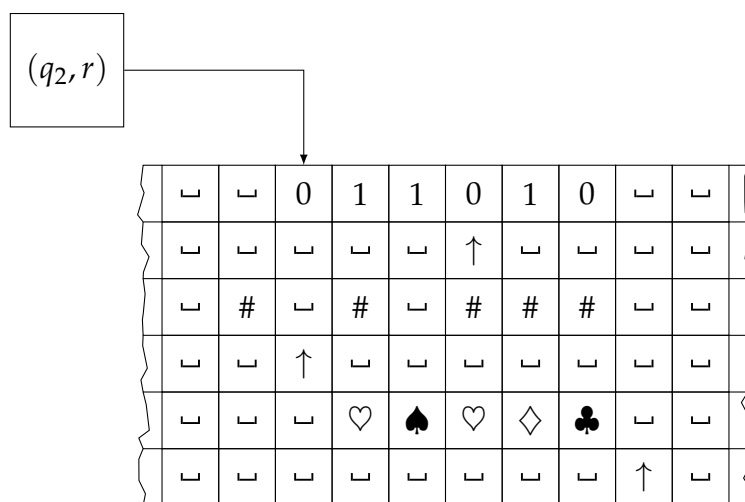


Figure 1.5: A 1-DTM with a multi-track tape can simulate any given multi-tape DTM. The odd numbered tracks represent the contents of the tapes of the DTM illustrated in Figure 1.4, while the even numbered tracks store the locations of its tape heads. The finite state control stores both the state of the DTM being simulated along with an additional component (represented by r in the figure) needed to carry out the simulation.

1. The 1-DTM has a tape with multiple tracks — which is equivalent to its tape alphabet being the Cartesian product of the alphabets corresponding to the individual tracks. The 1-DTM suggested in Figure 1.5 has two tracks on its tape for every tape of the DTM being simulated: one stores the contents of the corresponding tape and the other stores the position of the tape head corresponding to that tape.
2. Let us refer to the multi-tape DTM to be simulated as K . To simulate one step of K , our 1-DTM will make a complete pass over the region of its tape that includes all of the tape head markers. It remembers the symbols that the tape-heads of K scan in its finite control, determines the action of K through a finite-size table look-up, and then performs another scan to update the symbols and tape-head locations stored on the tracks of its own tape.

It is tedious to express such a simulation formally, and there are different specific ways this general idea could be carried out — but with enough time and motivation it is certainly possible to give a formal definition for a 1-DTM performing this simulation. The simulation is *quadratic* in the worst case: the n -th step of K

requires $O(n)$ steps of the 1-DTM running the simulation, so simulating n steps of K is possible with $O(n^2)$ steps of the 1-DTM.²

Input, work, and output tapes

The possibility for a multi-tape Turing machine to have a dedicated read-only input tape and write-only output tape was mentioned above. Here we will establish this convention in more precise terms.

Hereafter, for the remainder of the course, unless explicitly stated otherwise, every Turing machine we consider is a DTM having tapes of the following three types:

1. A single *input tape*, which is read-only. The input string is written on this tape at the start of the computation with special non-input symbols marking the left and right ends of the input string. The tape head may move freely to read the input symbols, but cannot write to the tape or move past the special symbols marking the ends of the input.
2. Some number of *work tapes*, which are ordinary read/write tapes that serve as the machine's work space during the computation. They begin computations filled with blanks.
3. A single *output tape*, which is write-only, with a tape head that only moves right when it writes a symbol and otherwise remains stationary. Symbols written to this tape represent the output of the computation, and once a symbol is written it is permanently part of the output.

The transition function of such a machine must naturally satisfy constraints that reflect limitations of the input and output tapes. The output tape can be omitted for decision problems, where it is not needed (though it is also common to interpret the single-bit outputs 1 and 0, written to an output tape, as indicating acceptance and rejection, respectively).

A few features of this convention are worth highlighting. First, as suggested previously, it makes sublinear space bounds meaningful, by only charging for tape squares visited on the work tapes and not for the input or output. DTMs of this form can also be composed in a fairly simple way, and overall provide us with a robust computational model. Finally, the model can be extended in a simple way

²The notations $O(n)$ and $O(n^2)$ are likely familiar, but are explained precisely in the next chapter for those not familiar with this notation.

to allow for *oracle queries* to a given language or function, as we will discuss later in the course.

Other Turing machine variants and computational models

The precise details of the Turing machine model turn out to be less important than one might initially imagine. Variations in the number of tapes, the tape alphabet, the specific form of the transition function, and many other aspects can be made without changing the class of decision problems that can be solved. Equivalences among different variants of Turing machines are most typically established through *simulations* — meaning that one Turing machine executes a process that replicates or mimics the step-by-step operation of another, like the one described above for simulating a multi-tape DTM with a 1-DTM.

Many other computational models — including λ -calculus, random access machine models, cellular automata, and more — turn out to be equivalent to Turing machines in this same sense. Though it can be tedious to go through the details, Turing machines can simulate all of these models, and each of these models can simulate Turing machines in return. The fundamental idea that Turing machine computations are representative of a fully general notion of computation is called the *Church–Turing thesis*. Here is one statement of this thesis — but understand that it is the idea rather than the exact choice of words that is important.

Church–Turing thesis: Any function that can be computed by a mechanical process can be computed by a Turing machine.

Note that this is not a mathematical statement that can be proved or disproved; it concerns an informal notion of what it means for a function to be “computed by a mechanical process,” and the Turing machine model may be seen as one attempt to precisely formalize such a notion. The thesis is broadly accepted, and there are no known counterexamples (so long as we disregard computational efficiency, which this form of the thesis does not mention).

1.4 Running time and space

Complexity theory is not principally concerned with what can and cannot be computed, but rather what can and cannot be computed *efficiently*. To even address the issue of efficiency we must establish numerical measures of the resource usage of Turing machines. The two most fundamental resource measures are *time* and *space*,

which correspond to the number of computational steps and the number of tape squares (on the work tapes only) that are visited, respectively.

Running time

The *running time* of a DTM measures how many computational steps it takes as a function of its input length, in the worst case. That is, for each input length n , we consider the maximum number of steps taken over all inputs of that length.

Definition 1.5. Let M be a DTM with input alphabet Σ . The *running time* of M is the function $t_M : \mathbb{N} \rightarrow \mathbb{N} \cup \{\infty\}$ defined by

$$t_M(n) = \max_{x \in \Sigma^n} \{\text{number of steps } M \text{ runs for on input } x\}. \quad (1.29)$$

This value is understood to be ∞ if M runs forever on some input of length n .

Space usage

For the definition of space usage, we assume the conventions described previously, where there exists a dedicated read-only input tape and (possibly) a write-only output tape. Space usage is measured by counting only the work-tape squares visited during a computation. (Neither the input tape nor the output tape contributes to space usage.)

Definition 1.6. Let M be a DTM with input alphabet Σ . The *space usage* of M is the function $s_M : \mathbb{N} \rightarrow \mathbb{N} \cup \{\infty\}$ defined by

$$s_M(n) = \max_{x \in \Sigma^n} \{\text{number of work-tape squares visited by } M \text{ on input } x\}. \quad (1.30)$$

As for time, the value is taken to be ∞ if M visits infinitely many work-tape squares on an input of length n .

Chapter 2

Classical complexity classes

In this chapter we will define several important classical complexity classes, including L, P, BPP, NP, PP, PSPACE, and EXP, and examine known relationships among these classes. These are not the only classical complexity classes we will encounter in this course, but this selection is a good start; additional classes will be defined later as needed.

2.1 Deterministic time and space

Let us begin with a couple of general complexity classes that correspond to the decision problems that can be decided by DTMs, subject to bounds on their running time or working space. Before getting to this, however, let us take just a moment to recall some standard asymptotic notation that will be used throughout the chapter and course.

Big-O. For functions $f, g : \mathbb{N} \rightarrow \mathbb{N}$, we write $f(n) = O(g(n))$ to mean that there exists a constant $c > 0$ such that $f(n) \leq cg(n)$ for all sufficiently large n . The notation $g(n) = \Omega(f(n))$ is equivalent to $f(n) = O(g(n))$, where f and g swap positions.

Little-o. For functions $f, g : \mathbb{N} \rightarrow \mathbb{N}$, we write $f(n) = o(g(n))$ to mean that, for *every* constant $c > 0$, we have $f(n) \leq cg(n)$ for all sufficiently large n . Assuming that g is eventually positive, this condition is equivalent to

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0. \quad (2.1)$$

The notation $g(n) = \omega(f(n))$ is equivalent to $f(n) = o(g(n))$.

In words, $f(n) = O(g(n))$ indicates that asymptotically f grows no faster than a constant times g , while $f(n) = o(g(n))$ indicates that asymptotically f grows strictly slower than any positive constant times g .

DTIME

Now we are ready to define the complexity class $\text{DTIME}(f)$ for a given function $f : \mathbb{N} \rightarrow \mathbb{N}$, which contains every decision problem that can be decided by a DTM running in time $O(f(n))$. Here this definition is stated specifically for promise problems, though traditionally it is defined as a class of languages.

Definition 2.1. Let $f : \mathbb{N} \rightarrow \mathbb{N}$ be a function. The complexity class $\text{DTIME}(f)$ consists of all promise problems $A = (A_{\text{yes}}, A_{\text{no}})$ for which there exists a DTM M satisfying the following two conditions:

1. The running time of M satisfies $t_M(n) = O(f(n))$.
2. M accepts every string $x \in A_{\text{yes}}$ and rejects every string $x \in A_{\text{no}}$.

Note that we sometimes also write $\text{DTIME}(f(n))$ in place of $\text{DTIME}(f)$, or substitute a different expression involving the variable n , which indicates a function of the input length to be taken as the function f . For example, $\text{DTIME}(n^2)$ means $\text{DTIME}(f)$ for $f(n) = n^2$ and $\text{DTIME}(2^{f(n)})$ means $\text{DTIME}(g)$ for $g(n) = 2^{f(n)}$. Also note that there is an implicit Big-O in this definition, as is standard. For example, $\text{DTIME}(n^2)$ and $\text{DTIME}(3n^2 + 5)$ are the same class.

While the complexity class $\text{DTIME}(f)$ is defined for every function $f : \mathbb{N} \rightarrow \mathbb{N}$, it is really only of interest from the viewpoint of computational complexity when f is reasonably well behaved. In particular, the so-called *gap theorem* implies that there are choices for the function f that are so bizarre that

$$\text{DTIME}(f(n)) = \text{DTIME}(2^{f(n)}), \quad (2.2)$$

despite the exponential gap between the two time bounds. Indeed, a similar equality holds for some choice of f when the exponential is replaced by any increasing computable function, such as a double or triple exponential. This might seem impossible at first glance, but what this theorem reveals is less about time-bounded complexity and more about the potential weirdness of functions engineered specifically to undermine the natural notion of running time for Turing machines.

The standard way to disregard such pathological time bounds, placing the focus on the resources required to solve problems, is to restrict our attention to functions that a DTM can compute within a comparable time budget.

Definition 2.2. A function $f : \mathbb{N} \rightarrow \mathbb{N}$ is *time constructible* if there exists a DTM that, on input 0^n , writes the binary representation of $f(n)$ on its output tape within $O(f(n))$ steps, for every $n \in \mathbb{N}$.

Every function that can reasonably be considered as a natural bound on running time is time constructible — including polynomials, exponentials, functions such as $n \log n$, products and sums of such functions, and many more. Hereafter, every function f appearing as an argument to $\text{DTIME}(f)$ in this course will be time constructible, which avoids the seemingly paradoxical conclusions of the gap theorem.

DSPACE

The class $\text{DSPACE}(f)$ is defined analogously to $\text{DTIME}(f)$, using the space measure s_M from Chapter 1 in place of the running time t_M . There is one added requirement this time, which is that we require our DTMs to always halt (which is automatic in the time-bounded setting, but not for space given our definition of space usage).

An essential point to keep in mind is that space is measured on the work tapes of a multi-tape DTM only: the read-only input tape and the write-only output tape do not contribute to space usage. Without this convention, no meaningful notion of *sublinear* space would exist, as simply reading an entire input having length n obviously requires visiting n tape squares. As it turns out, sublinear space bounds and corresponding classes such as $\text{DSPACE}(\log n)$ are not only interesting but fundamentally important in complexity theory.

Definition 2.3. Let $f : \mathbb{N} \rightarrow \mathbb{N}$ be a function. The complexity class $\text{DSPACE}(f)$ consists of all promise problems $A = (A_{\text{yes}}, A_{\text{no}})$ for which there exists a DTM M satisfying the following two conditions:

1. The space usage of M satisfies $s_M(n) = O(f(n))$.
2. M halts on every input, accepts every string $x \in A_{\text{yes}}$, and rejects every string $x \in A_{\text{no}}$.

As with $\text{DTIME}(f)$, there is an implicit Big-O in the notation, so $\text{DSPACE}(n^3)$ and $\text{DSPACE}(2n^3 + 57)$, for example, are the same class.

The analogous notion to time constructibility, but for space instead of time, is naturally called *space-constructibility*.

Definition 2.4. A function $f : \mathbb{N} \rightarrow \mathbb{N}$ is *space constructible* if there exists a DTM that, on input 0^n , writes the binary representation of $f(n)$ on its output tape while using at most $O(f(n))$ squares of work-tape space, for every $n \in \mathbb{N}$.

Every function that would naturally be considered as a space bound is space constructible, including $\log n$, polynomials and exponentials in n , sums and products of these functions, and so on. As with DTIME , every function f that appears as an argument to $\text{DSPACE}(f)$ in this course is space constructible.

Simple relationships

Let us note two simple relationships among these classes. First, if a DTM runs for t steps, it can visit no more than $O(t)$ squares on its work tapes. Consequently, the containment

$$\text{DTIME}(f) \subseteq \text{DSPACE}(f) \quad (2.3)$$

holds rather trivially, for every function f (including ones that are not time or space constructible).

Second, suppose M is a DTM running in space s_M that halts on all inputs. This means that M visits at most $s_M(n)$ work tape squares during its computation on inputs of length n , and therefore enters no more than

$$n \cdot 2^{O(s_M(n))} \quad (2.4)$$

distinct configurations. It cannot enter a configuration more than once by the assumption that it halts, and therefore (2.4) also bounds the running time of M . Consequently,

$$\text{DSPACE}(s) \subseteq \text{DTIME}(n \cdot 2^{O(s(n))}) \quad (2.5)$$

for every function s (again including ones that are not time or space constructible). In particular, if $s(n) = \Omega(\log(n))$, then we have

$$\text{DSPACE}(s) \subseteq \text{DTIME}(2^{O(s)}). \quad (2.6)$$

Time- and space-hierarchy theorems

We now state two of the few unconditional separation results known in classical complexity theory: the fact that giving a Turing machine sufficiently more time or space strictly increases its computational power.

Theorem 2.5 (Time hierarchy theorem). *Let $f, g : \mathbb{N} \rightarrow \mathbb{N}$ be time-constructible functions satisfying $f(n) = o(g(n) / \log(g(n)))$. Then*

$$\text{DTIME}(f) \subsetneq \text{DTIME}(g). \quad (2.7)$$

Theorem 2.6 (Space hierarchy theorem). *Let $f, g : \mathbb{N} \rightarrow \mathbb{N}$ be functions with g space-constructible, $g(n) = \Omega(\log n)$, and $f(n) = o(g(n))$. Then*

$$\text{DSpace}(f) \subsetneq \text{DSpace}(g). \quad (2.8)$$

For example, for every positive integer k the following relationships are true.

$$\begin{aligned} \text{DTIME}(n^k) &\subsetneq \text{DTIME}(n^{k+1}) \\ \text{DSpace}((\log n)^k) &\subsetneq \text{DSpace}((\log n)^{k+1}) \end{aligned} \quad (2.9)$$

To convey the idea behind these theorems, let us give a high-level sketch of the proof of the time-hierarchy theorem that brushes some details aside. Fix a time-constructible function h , and let $\langle M \rangle \in \{0, 1\}^*$ denote a binary encoding of a DTM M (according to some reasonable way of doing this encoding). Then define a DTM K having input alphabet $\{0, 1\}$ like so.

On input $w \in \{0, 1\}^*$, K operates as follows:

1. If w does not have the form $w = \langle M \rangle 01^k$ for a DTM M with input alphabet $\{0, 1\}$ and some $k \in \mathbb{N}$, then reject.
2. Compute $t = h(|w|)$.
3. Simulate M on input w for t steps. If M rejects within t steps, then accept; otherwise reject.

The idea is that K decides some language, conventionally denoted $L(K)$. This language is evidently contained in $\text{DTIME}(g)$ for g being the running time of K , which depends on precisely how the simulation in step 3 is performed. Note also that the

time-constructibility of h implies that step 2 can be performed in time $O(h(n))$, which is relevant to analyzing the running time of K .

On the other hand, by the specific way that K is defined, it cannot be that $L(K) \in \text{DTIME}(f)$ for $f(n) = o(h(n))$. To see this, assume toward contradiction that there does exist a DTM M that decides $L(K)$ in time $o(h(n))$. This implies that there must exist a natural number n_0 such that, for all $n \geq n_0$, the DTM M halts on all inputs of length n in strictly fewer than $h(n)$ steps. If we choose k to be large enough so that the string $w = \langle M \rangle 01^k$ satisfies $|w| \geq n_0$, then we find that

$$w \in L(K) \Leftrightarrow w \notin L(M). \quad (2.10)$$

(This, by the way, is why the input takes this unusual form $\langle M \rangle 01^k$; it allows the input to encode a DTM of interest and to be as long as needed to allow the asymptotic behaviour of the function h to kick in.) In short, K simulates M on input w , with enough time to complete the simulation because M runs for fewer than $h(n)$ steps. Critically, K answers *opposite* to the way M answers (i.e., if M accepts, then K rejects; and if M rejects, then K accepts). This contradicts the assumption that M decides $L(K)$, so we conclude that no DTM whose running time is $o(h(n))$ can decide $L(K)$.

The underlying technique on display in this proof is a form of *diagonalization*, which we will see again later in the course. By carefully choosing h and performing the simulation in step 3 in the description of K as efficiently as possible, the time-hierarchy theorem can be formally proved as it is stated. The space-hierarchy theorem is proved by essentially the same argument, with running time replaced by space usage and the simulation of step 3 carried out under a space bound rather than a time bound.

Four deterministic resource-bounded classes

Next we define four complexity classes: two of them are time-bounded, namely P (polynomial time) and EXP (exponential time), while the other two are space-bounded, namely L (logarithmic space) and PSPACE (polynomial space). According to our conventions, these are classes of promise problems, though they are commonly restricted to languages. (Recall that a language-decision problem $L \subseteq \Sigma^*$ is identified with the promise problem $(L, \Sigma^* \setminus L)$, for which the promise is trivial.)

Definition 2.7. The class P (polynomial time) is defined as

$$P = \bigcup_{k \geq 1} \text{DTIME}(n^k). \quad (2.11)$$

Equivalently, a promise problem A belongs to P if there exists a DTM M that decides A and a polynomial p such that $t_M(n) \leq p(n)$ for all $n \in \mathbb{N}$.

Definition 2.8. The class EXP (exponential time) is defined as

$$\text{EXP} = \bigcup_{k \geq 1} \text{DTIME}(2^{n^k}). \quad (2.12)$$

Equivalently, $A \in \text{EXP}$ if there exists a DTM M that decides A and a polynomial p such that $t_M(n) \leq 2^{p(n)}$ for all $n \in \mathbb{N}$.

Definition 2.9. The class L (logarithmic space) is defined as

$$L = \text{DSpace}(\log n). \quad (2.13)$$

Equivalently, $A \in L$ if there exists a DTM M that decides A and a constant $c > 0$ such that $s_M(n) \leq c \log n$ for all sufficiently large n .

Definition 2.10. The class PSPACE (polynomial space) is defined as

$$\text{PSPACE} = \bigcup_{k \geq 1} \text{DSpace}(n^k). \quad (2.14)$$

Equivalently, $A \in \text{PSPACE}$ if there exists a DTM M that decides A and a polynomial p such that $s_M(n) \leq p(n)$ for all $n \in \mathbb{N}$.

These four classes satisfy the inclusions

$$L \subseteq P \subseteq \text{PSPACE} \subseteq \text{EXP}, \quad (2.15)$$

which follow from the relationships between deterministic time and space covered in Section 2.1 above. In particular, the inclusion $P \subseteq \text{PSPACE}$ is immediate from $\text{DTIME}(f) \subseteq \text{DSpace}(f)$, while the other two follow from $\text{DSpace}(s) \subseteq \text{DTIME}(2^{O(s)})$ when $s(n) = \Omega(\log(n))$.

The hierarchy theorems imply that $L \subsetneq \text{PSPACE}$ and $P \subsetneq \text{EXP}$. Therefore, at least one of the three adjacent inclusions in (2.15) is strict — and indeed they are all widely conjectured to be strict. However, no proof of strictness is known for any of them, and it could even be that $L = P$ and $\text{PSPACE} = \text{EXP}$ are true simultaneously.

2.2 Complexity classes via quantifiers

The remaining complexity classes to be defined in this chapter are NP, BPP, and PP. The class NP, short for *nondeterministic polynomial time*, is based on the notion of nondeterminism, while BPP and PP, short for *bounded-error probabilistic polynomial time* and *probabilistic polynomial time*, respectively, are based on randomized computations. It is possible to define nondeterministic and probabilistic versions of the Turing machine model, through which these classes have natural definitions, but we will take a different route involving the use of quantifiers over strings. This allows us to work purely with the DTM model introduced in Chapter 1. Later on in the course we will return to the same quantifier-based method to define additional complexity classes of interest.

Pairing and tuple functions

We must first select a *pairing function*, which is a way of encoding two strings into one. There are any number of specific ways that this can reasonably be done, and which one we choose ultimately has little importance — but the simplest path forward is to just choose one rather than talking about them in general.

Let us assume first that we are working with the binary alphabet, and we have two binary strings x and y that we wish to encode as a single binary string, which will be denoted $\langle x, y \rangle$. We require that it is possible to recover both x and y from $\langle x, y \rangle$, and that it is computationally easy both to perform these recoveries as well as compute $\langle x, y \rangle$ given x and y . Here is a simple way to do this that proceeds in two steps.

First, we introduce a new symbol $\#$ distinct from 0 and 1, and combine the pair (x, y) of binary strings into the single string

$$x\#y \in \{0, 1, \#\}^* \quad (2.16)$$

over a three-symbol alphabet. Forming this string given x and y is easy, as is recovering the strings x and y from it — these are simple string-processing operations. The problem is that we want the encoded string to be over the binary alphabet.

The second step solves this problem, taking us back to the binary alphabet. Specifically, we apply the encoding

$$0 \mapsto 00, \quad 1 \mapsto 01, \quad \# \mapsto 10 \quad (2.17)$$

to each symbol of $x\#y$ and concatenate the results in order. The result is the binary string $\langle x, y \rangle$. Recovering x and y from $\langle x, y \rangle$ is a matter of scanning from left to right

and decoding each pair of symbols. Both encoding and decoding can be performed in linear time and constant space on a DTM.

The scheme extends without difficulties to tuples of more than two strings: for any sequence $(x_1, \dots, x_n)$ of binary strings, we form the string

$$x_1 \# x_2 \# \dots \# x_n \tag{2.18}$$

and apply the same encoding as before to obtain $\langle x_1, \dots, x_n \rangle$. We can also easily extend this scheme to strings over different alphabets in place of the binary alphabet by choosing a suitable symbol encoding in place of (2.17), always taking $\#$ to be an additional symbol beyond those appearing in $x_1, \dots, x_n$. To be clear, our assumption will always be that $\langle x_1, \dots, x_n \rangle$ is a binary string, regardless of n and the alphabets for the strings $x_1, \dots, x_n$.

Note that this encoding method is one-to-one but not onto: for distinct tuples of strings over fixed alphabets, the encodings are always distinct binary strings, but not every binary string arises as an encoding of a tuple. This will cause us no difficulties: strings that fail to encode pairs or tuples of strings are easily recognizable and, depending on the situation, can either be rejected or treated as “don’t care” inputs in promise problems. There are alternative pairing functions that are both one-to-one and onto, but we will not require their use.

With this convention in place, we are free to consider computations taking multiple input strings. For example, if M is a DTM and we refer to running M on an input of the form (x, y) or $(x_1, \dots, x_n)$, we mean that M is run on $\langle x, y \rangle$ or $\langle x_1, \dots, x_n \rangle$, respectively.

Polynomial resource bounds

A *polynomial resource bound* is a non-decreasing function of the form $p : \mathbb{N} \rightarrow \mathbb{N}$ for which there exists a polynomial-time DTM that outputs $0^{p(n)}$ on input 0^n for every $n \in \mathbb{N}$. Polynomial resource bounds are therefore polynomially bounded (i.e., $p(n) = O(n^c)$ for some constant $c \geq 1$) and are computable in polynomial time. In essence, this is a simple adaptation and relaxation of time constructibility that will be convenient for our purposes. Every polynomial with natural number coefficients is a polynomial resource bound, but polynomial resource bounds need not be polynomials in the usual algebraic sense.

It will be very typical in this course that an input string x is understood to be fixed or under consideration. Unless it is explicitly stated otherwise, we will reserve the letter n for the length of the input string. Along similar lines, if p is

a given polynomial resource bound, we will typically write p to mean $p(n)$, as polynomial resource bounds are almost always evaluated for the input length. In short, when x is understood to be the input string and p is a polynomial resource bound, we write

$$n = |x| \quad \text{and} \quad p = p(n) = p(|x|) \quad (2.19)$$

for the sake of brevity. (There are a couple of occasions when a resource bound is evaluated on a different input — such as the length $|\langle x, y \rangle|$ of an encoded pair of strings — and in such cases the argument will be written explicitly.)

A “for most” quantifier

One further piece of notation will be convenient. Two of the three classes defined below place a condition on the *number* of strings satisfying some predicate rather than on the existence of one, and these counting conditions are conveniently expressed through a quantifier $\mathfrak{A}$, which may be read as “for most.” It is sometimes called the Zachos–Heller quantifier, named for Stathis Zachos and Hans Heller who introduced it in the early 1980s (using a different symbol).

Supposing that m is a positive integer and $P : \{0, 1\}^m \rightarrow \{0, 1\}$ is a predicate, we write

$$(\mathfrak{A} x \in \{0, 1\}^m) [P(x)] \quad (2.20)$$

to mean that P holds for at least two thirds of all strings $x \in \{0, 1\}^m$, or in other words that

$$|\{x \in \{0, 1\}^m : P(x)\}| \geq \frac{2}{3} \cdot 2^m. \quad (2.21)$$

More generally, for a real number $c \in [0, 1]$ we write

$$(\mathfrak{A}^c x \in \{0, 1\}^m) [P(x)] \quad (2.22)$$

to mean that

$$|\{x \in \{0, 1\}^m : P(x)\}| \geq c \cdot 2^m, \quad (2.23)$$

so that $\mathfrak{A}$ abbreviates $\mathfrak{A}^{2/3}$.

The class NP

Now we are prepared to define the remaining complexity classes for the chapter, beginning with NP: the class of promise problems admitting an efficiently verifiable yes-certificate.

Definition 2.11. A promise problem $A = (A_{\text{yes}}, A_{\text{no}})$ over an alphabet Σ is contained in NP if there exist a polynomial resource bound p and a language $B \in \mathcal{P}$ such that these implications are true.

$$x \in A_{\text{yes}} \implies (\exists y \in \{0,1\}^p) [\langle x, y \rangle \in B] \quad (2.24)$$

$$x \in A_{\text{no}} \implies (\forall y \in \{0,1\}^p) [\langle x, y \rangle \notin B] \quad (2.25)$$

The language B in this definition, which happens to be a language over the binary alphabet given our pairing function assumption, plays the role of a *verifier* for the problem A . That is, given an input $x \in \Sigma^*$ and a candidate binary string y of length $p = p(|x|)$, the condition $\langle x, y \rangle \in B$ means that y is a valid *witness* that x is a yes-input. For x to be a yes-input, at least one such witness y must exist, while no such y can exist when x is a no-input. As p is a polynomial resource bound, witnesses are polynomial-length strings in this definition.

The name NP is short for *nondeterministic polynomial time*, which refers to an alternative formulation of this class in terms of nondeterministic Turing machines that run in polynomial time. A nondeterministic Turing machine is one whose transition function associates each state/symbol pair with a set of possible actions rather than a single one. This leads to multiple computation paths being available depending on which of the possible actions are performed, and the machine is said to *accept* the input if there exists at least one path that reaches the accept state. This nondeterministic formulation can be shown to be equivalent to the quantifier-based definition above.

The class BPP

The next class we define is BPP, which is short for *bounded error probabilistic polynomial time*. It represents a natural complexity theoretic way of classifying the promise problems that can be efficiently decided in polynomial time using randomness. The term *bounded error* can be interpreted as a requirement that a randomized polynomial-time computation provides an output that is strongly biased to the correct answer, but not necessarily perfect. We define the class more precisely using the $\mathcal{P}$ quantifier as follows.

Definition 2.12. A promise problem $A = (A_{\text{yes}}, A_{\text{no}})$ over an alphabet Σ is contained in BPP if there exist a polynomial resource bound p and a language $B \in \mathcal{P}$ such that these implications are true.

$$x \in A_{\text{yes}} \implies (\mathcal{P} y \in \{0,1\}^p) [\langle x, y \rangle \in B] \quad (2.26)$$

$$x \in A_{\text{no}} \implies (\mathcal{P} y \in \{0,1\}^p) [\langle x, y \rangle \notin B] \quad (2.27)$$

The natural interpretation of this definition is that the string y represents a random bit string, which provides all the randomness we need — and the requirements read that *most* random strings lead us to the correct answer. The specific constants $2/3$ and $1/3$ in the definition of the $\mathfrak{R}$ quantifier are not special; any constants $\alpha, \beta \in (0, 1)$ with $\alpha > \beta$ in place of $2/3$ and $1/3$ give us the same class. These bounds can also be replaced by a wide range of functions of the input length, not just constants. It is instructive to see how this works, as we will generalize the same approach to multiple quantum classes later on.

The essential idea is to consider a simple but powerful error-reduction method: rather than randomly choosing a single random string $y \in \{0, 1\}^p$ and checking whether or not $\langle x, y \rangle \in B$, choose several candidate strings $y_1, \dots, y_N \in \{0, 1\}^p$, where N could be polynomially large in the input length, and count for how many of these strings satisfy $\langle x, y_k \rangle \in B$. The final decision is to accept if the count exceeds a particular threshold and reject otherwise. A well-known technical tool for analyzing the resulting error probability is the following theorem.

Theorem 2.13 (Hoeffding's inequality). *Let $X_1, \dots, X_N$ be independent random variables taking values in $[0, 1]$, and let $\mu = \frac{1}{N} \sum_{i=1}^N \mathbb{E}(X_i)$. For every $\varepsilon > 0$,*

$$\Pr\left(\frac{1}{N} \sum_{i=1}^N X_i \geq \mu + \varepsilon\right) \leq e^{-2N\varepsilon^2}, \quad (2.28)$$

$$\Pr\left(\frac{1}{N} \sum_{i=1}^N X_i \leq \mu - \varepsilon\right) \leq e^{-2N\varepsilon^2}. \quad (2.29)$$

From this theorem we can obtain the following lemma, which offers a convenient formulation through which the error-reduction method above can be analyzed directly.

Lemma 2.14 (Amplification lemma). *Let $\alpha, \beta \in [0, 1]$ satisfy $\alpha > \beta$, let $r > 0$ be a positive real number, and let N be an integer satisfying*

$$N \geq \frac{2r}{(\alpha - \beta)^2}. \quad (2.30)$$

Assume $X_1, \dots, X_N$ are independent random variables taking values in $[0, 1]$, each having expectation μ . We have the following implications.

$$\mu \geq \alpha \implies \Pr\left(X_1 + \dots + X_N \leq \frac{\alpha + \beta}{2} \cdot N\right) \leq 2^{-r} \quad (2.31)$$

$$\mu \leq \beta \implies \Pr\left(X_1 + \dots + X_N \geq \frac{\alpha + \beta}{2} \cdot N\right) \leq 2^{-r}. \quad (2.32)$$

Proof. Suppose first that $\mu \geq \alpha$, which implies that

$$\frac{\alpha + \beta}{2} = \alpha - \frac{\alpha - \beta}{2} \leq \mu - \frac{\alpha - \beta}{2}. \quad (2.33)$$

Therefore, the second inequality of Theorem 2.13, applied with $\varepsilon = (\alpha - \beta)/2$, together with (2.30), gives

$$\Pr\left(\frac{1}{N} \sum_{i=1}^N X_i \leq \frac{\alpha + \beta}{2}\right) \leq e^{-N(\alpha - \beta)^2/2} \leq e^{-r} \leq 2^{-r}. \quad (2.34)$$

The case $\mu \leq \beta$ is similar, using the first inequality of Theorem 2.13 rather than the second. $\square$

The threshold $(\alpha + \beta)/2$ is simply the midpoint of the two probability bounds being distinguished, which makes the two cases symmetric. Note that the bound (2.30) on the number of repetitions is not the best possible (the exponent can be improved by a factor of $\ln 2$), but it is convenient and easily remembered. For the particular case $\alpha = 2/3$ and $\beta = 1/3$ the lemma requires $N \geq 18r$ repetitions.

Every application of the amplification lemma in this course concerns independent random variables taking the values 0 and 1, which admits the following purely algebraic form of the amplification lemma. For a binary string y we write $|y|_1$ to denote the *Hamming weight* of y , which is the number of 1s it contains.

Corollary 2.15. *Let $\alpha, \beta \in [0, 1]$ satisfy $\alpha > \beta$, let $r > 0$ be a positive real number, let N be an integer satisfying (2.30), and let $t = \frac{\alpha + \beta}{2} \cdot N$. For every real number $\mu \in [0, 1]$ the following implications are true.*

$$\mu \geq \alpha \implies \sum_{\substack{y \in \{0,1\}^N \\ |y|_1 \geq t}} \mu^{|y|_1} (1 - \mu)^{N - |y|_1} \geq 1 - 2^{-r} \quad (2.35)$$

$$\mu \leq \beta \implies \sum_{\substack{y \in \{0,1\}^N \\ |y|_1 \geq t}} \mu^{|y|_1} (1 - \mu)^{N - |y|_1} \leq 2^{-r} \quad (2.36)$$

Proof. Let $X_1, \dots, X_N$ be independent random variables, each taking the value 1 with probability μ and the value 0 with probability $1 - \mu$, so that each takes values in $[0, 1]$ and has expectation μ . For each string $y \in \{0, 1\}^N$ the probability that $X_1, \dots, X_N$ take the values $y_1, \dots, y_N$ is $\mu^{|y|_1} (1 - \mu)^{N - |y|_1}$, so the sum appearing in both implications is equal to

$$\Pr(X_1 + \dots + X_N \geq t). \quad (2.37)$$

The implications of the corollary follow from (2.31) and (2.32) of Lemma 2.14. $\square$

Error reduction for BPP is now a matter of repeating the underlying computation and taking a majority.

Theorem 2.16 (Error reduction for BPP). *Let $A = (A_{\text{yes}}, A_{\text{no}})$ be a promise problem contained in BPP and let r be a polynomial resource bound. There exist a polynomial resource bound q and a language $C \in \mathcal{P}$ such that these implications are true, where $\varepsilon = \varepsilon(n) = 2^{-r(n)}$.*

$$x \in A_{\text{yes}} \implies (\mathfrak{R}^{1-\varepsilon} y \in \{0,1\}^q) [\langle x, y \rangle \in C] \quad (2.38)$$

$$x \in A_{\text{no}} \implies (\mathfrak{R}^{1-\varepsilon} y \in \{0,1\}^q) [\langle x, y \rangle \notin C] \quad (2.39)$$

Proof. Let p and B be as in Definition 2.12 for the problem A , let $N = 18r$, let $q = Np$, and identify each string $y \in \{0,1\}^q$ with a tuple $(y_1, \dots, y_N)$ of strings $y_1, \dots, y_N \in \{0,1\}^p$. Define a language C as

$$C = \left\{ \langle x, y \rangle : \left| \{k \in \{1, \dots, N\} : \langle x, y_k \rangle \in B\} \right| \geq \frac{N}{2} \right\}, \quad (2.40)$$

which is contained in $\mathcal{P}$ because B is and because N is a polynomial resource bound.

Now fix an input x , and for each $k \in \{1, \dots, N\}$ let X_k be the random variable taking the value 1 if $\langle x, y_k \rangle \in B$ and the value 0 otherwise, for $y_1, \dots, y_N \in \{0,1\}^p$ chosen independently and uniformly at random. These random variables are independent, take values in $[0, 1]$, and each has expectation

$$\mu = 2^{-p} |\{y \in \{0,1\}^p : \langle x, y \rangle \in B\}|, \quad (2.41)$$

which is the probability that the original computation accepts x . By (2.40), the string $\langle x, y \rangle$ is contained in C if and only if the average of $X_1, \dots, X_N$ is at least $1/2$, which is the midpoint of $2/3$ and $1/3$. Definition 2.12 gives $\mu \geq 2/3$ when $x \in A_{\text{yes}}$ and $\mu \leq 1/3$ when $x \in A_{\text{no}}$, so Lemma 2.14, applied with $\alpha = 2/3$ and $\beta = 1/3$, bounds the probability of an incorrect answer by 2^{-r} in both cases. $\square$

The same argument can be applied to other constants in place of $2/3$ and $1/3$, which may require changing the number of repetitions as well as the threshold value for acceptance, but not the final conclusion. Indeed, as was suggested previously, α and β can be replaced by a wide range of functions in place of constants — but we will brush this aside for now and revisit it later in Chapter 5 in the context of the complexity class BQP.

Here is a simple, but nevertheless very interesting, implication of the previous theorem, which follows from the union bound.

Corollary 2.17. *Let $A = (A_{\text{yes}}, A_{\text{no}})$ be a promise problem over an alphabet Σ contained in BPP. There exist a polynomial resource bound q , a language $C \in \mathcal{P}$, and a family of strings $\{y_n : n \in \mathbb{N}\}$ with $|y_n| = q(n)$ for every n , such that*

$$x \in A_{\text{yes}} \implies \langle x, y_n \rangle \in C \quad (2.42)$$

$$x \in A_{\text{no}} \implies \langle x, y_n \rangle \notin C \quad (2.43)$$

for every $n \in \mathbb{N}$ and every string x of length n .

Proof. Taking $r(n) = n|\Sigma| + 1$ in Theorem 2.16, we conclude that a polynomial resource bound q and a language $C \in \mathcal{P}$ exist for which (2.38) and (2.39) are true for $\varepsilon = 2^{-n|\Sigma|-1}$.

For each natural number $n \in \mathbb{N}$, and for each string $x \in \Sigma^n \cap (A_{\text{yes}} \cup A_{\text{no}})$, define

$$Y_x = \begin{cases} \{y \in \{0,1\}^{q(n)} : \langle x, y \rangle \notin C\} & x \in A_{\text{yes}} \\ \{y \in \{0,1\}^{q(n)} : \langle x, y \rangle \in C\} & x \in A_{\text{no}}. \end{cases} \quad (2.44)$$

In words, Y_x is the subset of random bit-strings in $\{0,1\}^q$ leading to an incorrect answer on input x . It is the case that

$$|Y_x| \leq 2^{-n|\Sigma|-1} 2^{q(n)}. \quad (2.45)$$

As there are at most $|\Sigma|^n$ strings of length n , and $|\Sigma| \leq 2^{|\Sigma|}$, we have

$$\left| \bigcup_x Y_x \right| \leq \sum_x |Y_x| \leq |\Sigma|^n \cdot 2^{-n|\Sigma|-1} \cdot 2^{q(n)} \leq \frac{2^{q(n)}}{2}, \quad (2.46)$$

where both the union and the sum range over the strings x of length n satisfying the promise.

Thus, for each $n \in \mathbb{N}$, at most half of the strings $y \in \{0,1\}^{q(n)}$ are contained in one or more of the sets Y_x ranging over all $x \in \Sigma^n \cap (A_{\text{yes}} \cup A_{\text{no}})$ — which is to say that at most half of these strings ever lead to a wrong answer on any input of length n . Taking $y_n \in \{0,1\}^{q(n)}$ to be any string not contained in any of the sets Y_x completes the proof. $\square$

Intuitively speaking, this corollary shows that bounded-error probabilistic algorithms can be made so reliable that a single choice of random string works simultaneously for every input of a given length. It is important to recognize, however, that the corollary does not provide a method to compute these strings; it only proves that they exist.

The class PP

The class PP is defined in a similar way to BPP, but with a considerably weaker separation between yes- and no-inputs.

Definition 2.18. A promise problem $A = (A_{\text{yes}}, A_{\text{no}})$ over an alphabet Σ is contained in PP if there exist a polynomial resource bound p and a language $B \in \mathcal{P}$ such that these implications are true.

$$x \in A_{\text{yes}} \implies |\{y \in \{0,1\}^p : \langle x, y \rangle \in B\}| > \frac{1}{2} \cdot 2^p \quad (2.47)$$

$$x \in A_{\text{no}} \implies |\{y \in \{0,1\}^p : \langle x, y \rangle \in B\}| < \frac{1}{2} \cdot 2^p \quad (2.48)$$

The name PP stands for *probabilistic polynomial time*. Intuitively speaking, membership in PP requires only that the underlying probabilistic computation is correct on a strict majority of the random binary strings it might draw, as opposed to a robust majority like for BPP. The absence of a nonnegligible gap between the two thresholds is highly significant, and in particular renders the error reduction method for BPP ineffective.

Note that we could alternatively define a majority quantifier and define PP along similar lines to our definition for BPP, but we will not need to make use of such a quantifier in this course so we do not bother.

It should be noted that the inequality in (2.47), but not (2.48), is often taken to be a non-strict inequality when defining this class — but whether the inequality is strict or not strict does not change the class itself, and taking both inequalities to be strict is more convenient for our purposes. From the definition it follows trivially that PP is closed under complementation:

$$(A_{\text{yes}}, A_{\text{no}}) \in \text{PP} \iff (A_{\text{no}}, A_{\text{yes}}) \in \text{PP}. \quad (2.49)$$

The class PP is widely believed to be larger than BPP. For example, NP is contained in PP, but is widely conjectured not to be contained in BPP. (Arguing that $\text{NP} \subseteq \text{PP}$ makes for a good exercise.) We will have more to say about PP in a few chapters, where we will find that so-called *GapP functions* offer a very convenient tool for reasoning about this class and its relationship to quantum complexity classes.

2.3 Relationships and the containment diagram

We will conclude the chapter with a summary of known containments among the complexity classes introduced in this chapter.

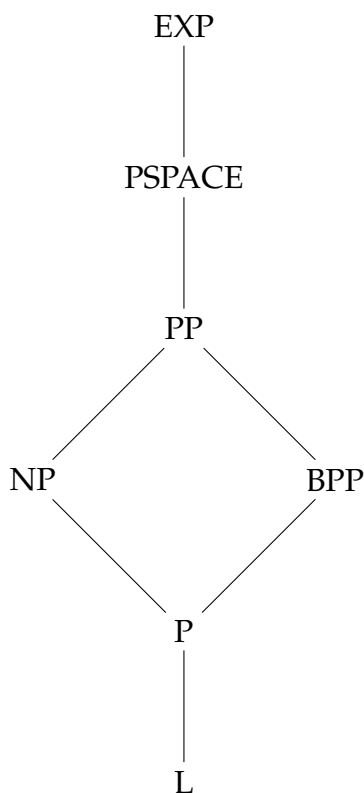


Figure 2.1: A so-called *Hasse diagram* showing relationships among the complexity classes defined in this chapter. Each class is contained in every other class that can be reached by moving upward along one or more edges.

We have already observed that

$$L \subseteq P \subseteq PSPACE \subseteq EXP. \quad (2.50)$$

Among the remaining containments illustrated in the diagram, we have

$$P \subseteq NP, \quad P \subseteq BPP, \quad \text{and} \quad BPP \subseteq PP, \quad (2.51)$$

which are all immediate (or nearly so) from the definitions. The remaining ones are

$$NP \subseteq PP \quad \text{and} \quad PP \subseteq PSPACE. \quad (2.52)$$

As suggested above, the first one is a good exercise. For the second, taking p and B as in Definition 2.18, a polynomial-space machine can iterate through all $y \in \{0,1\}^p$, testing whether $\langle x, y \rangle \in B$ and maintaining a running count of the accepting strings. Comparing the final count against $\frac{1}{2} \cdot 2^p$ decides the answer.

Chapter 3

Reductions, completeness, and Boolean circuits

This is the final chapter of the classical complexity theory portion of the course. We will begin by defining polynomial-time computable functions, as well as the more restricted class of logarithmic-space computable functions (or *log-space* computable functions, for short). With these classes of functions in hand, we then move on to the notion of *reductions* between promise problems — specifically, polynomial-time reductions (also called *Karp reductions*) and log-space reductions. This leads to the notions of *NP-completeness* and *P-completeness*, which can be extended to other classes as well. The Boolean circuit model is then introduced, along with its connections to Turing machines. Finally, the chapter finishes with a short discussion of the Cook–Levin theorem, which identifies 3SAT as an NP-complete language, opening the door to a vast catalogue of known NP-complete problems.

3.1 Polynomial-time and log-space computable functions

This section introduces two classes of functions — namely polynomial-time computable functions and log-space computable functions — which will be used to define *reductions* between problems in the next section.

Polynomial-time computable functions

Let us begin with polynomial-time computable functions, which are defined as follows.

Definition 3.1. Let Σ and Γ be alphabets. A function $f : \Sigma^* \rightarrow \Gamma^*$ is *polynomial-time computable* if there exists a DTM M running in polynomial time that, when run on each possible input string $x \in \Sigma^*$, writes the string $f(x) \in \Gamma^*$ to its output tape.

Note that $|f(x)|$ must be polynomially bounded in $|x|$ for any polynomial-time computable function, as a DTM cannot output more symbols than the total number of steps for which it runs.

Polynomial-time computable *predicates* are polynomial-time computable functions of the form

$$V : \Sigma^* \rightarrow \{0, 1\}. \quad (3.1)$$

They provide a simple functional characterization of the class P, which is that a promise problem $A = (A_{\text{yes}}, A_{\text{no}})$ is in P if and only if there exists a polynomial-time computable predicate V satisfying $V(x) = 1$ for every $x \in A_{\text{yes}}$ and $V(x) = 0$ for every $x \in A_{\text{no}}$.

The class of polynomial-time computable functions is closed under composition. That is, if

$$f : \Sigma^* \rightarrow \Gamma^* \quad \text{and} \quad g : \Gamma^* \rightarrow \Delta^* \quad (3.2)$$

are both polynomial-time computable, then so is

$$g \circ f : \Sigma^* \rightarrow \Delta^*. \quad (3.3)$$

In particular, $f(x)$ can be computed from x in time polynomial in $|x|$, and $g(f(x))$ can be computed from $f(x)$ in time polynomial in $|f(x)|$, which is also polynomial in $|x|$ because $|f(x)|$ is polynomial in $|x|$. This closure property makes polynomial-time reductions (which are defined in the next section) transitive.

Log-space computable functions

Log-space computable functions are defined analogously to polynomial-time computable functions, but with a logarithmic bound on the work-tape space usage rather than a polynomial bound on running time.

Definition 3.2. Let Σ and Γ be alphabets. A function $f : \Sigma^* \rightarrow \Gamma^*$ is *log-space computable* if there exists a DTM M running in logarithmic space such that M halts with $f(x)$ written on its output tape for every input string $x \in \Sigma^*$.

Notice that every log-space computable function f is polynomial-time computable, based on the same reasoning used in Section 2.1 to establish the relationship

$$\text{DSPACE}(s) \subseteq \text{DTIME}(2^{O(s)}). \quad (3.4)$$

On the other hand, because the output tape of a DTM does not contribute to its space usage, it is possible that $|f(x)|$ exceeds $O(\log n)$ for a log-space computable function.

Log-space computable functions are, like polynomial-time computable functions, closed under composition. That is, if

$$f : \Sigma^* \rightarrow \Gamma^* \quad \text{and} \quad g : \Gamma^* \rightarrow \Delta^* \quad (3.5)$$

are log-space computable, then so is

$$g \circ f : \Sigma^* \rightarrow \Delta^*. \quad (3.6)$$

This is not obvious, and proving it requires a trick. Naively composing $g \circ f$ by first computing $f(x)$ and then computing g on the result would require storing the intermediate string $f(x)$, which generally is not possible in logarithmic space. The standard workaround is typical of space-bounded computation: each symbol of $f(x)$ is re-computed, possibly over and over, as needed to compute g . In effect, the computations of the two functions are therefore interleaved to compute the composition, rather than one computation being performed after the other. Working out the details is a good technical exercise to gain familiarity with space-bounded computation.

3.2 Reductions and completeness

Reductions formalize the intuitive notion of one problem being “no harder” than another. There are many sorts of reductions, but we will restrict our attention to two basic ones: *Karp reductions* (also called *polynomial-time many-one reductions*) and *log-space reductions*. They correspond naturally to the two computational function classes introduced in the previous section, and both are examples of *mapping* or *many-one* reductions. In particular, they transform inputs of a source problem into inputs of a target problem, with the requirement that yes and no inputs of the source problem always map to inputs of the target problem sharing the same yes/no answer.

Karp reductions and NP-completeness

Karp reductions are the most common type of reduction between promise problems (most often languages).

Definition 3.3. Let $A = (A_{\text{yes}}, A_{\text{no}})$ and $B = (B_{\text{yes}}, B_{\text{no}})$ be promise problems over alphabets Σ and Γ respectively. We say that A *Karp reduces* to B , written

$$A \leq_m^P B, \quad (3.7)$$

if there exists a polynomial-time computable function $f : \Sigma^* \rightarrow \Gamma^*$ such that

$$x \in A_{\text{yes}} \implies f(x) \in B_{\text{yes}}, \quad (3.8)$$

$$x \in A_{\text{no}} \implies f(x) \in B_{\text{no}}. \quad (3.9)$$

The function f is called a *Karp reduction* from A to B .

Note that the definition places no requirement on what the function f outputs on strings outside $A_{\text{yes}} \cup A_{\text{no}}$. That is, *don't care* inputs can be mapped to any string, whether inside or outside $B_{\text{yes}} \cup B_{\text{no}}$.

The name *Karp reduction* honours Richard Karp, a pioneer of the theory of NP-completeness. With Karp reductions in hand, we can proceed directly to the definitions of hardness and completeness for any complexity class.

Definition 3.4. Let $\mathcal{C}$ be a set of promise problems (i.e., a complexity class). A promise problem B is $\mathcal{C}$ -*hard* under Karp reductions if $A \leq_m^P B$ for every promise problem $A \in \mathcal{C}$. A promise problem B is $\mathcal{C}$ -*complete* if $B \in \mathcal{C}$ and B is $\mathcal{C}$ -hard.

The intuition is that problems complete for a class $\mathcal{C}$ are the *hardest* problems in $\mathcal{C}$ in the sense that every problem in $\mathcal{C}$ Karp-reduces to any $\mathcal{C}$ -complete problem. This definition can also be extended to other reductions, such as log-space reductions, which are discussed in the next subsection.

Some complexity classes have complete problems and some do not. For example, NP and PSPACE have many known complete problems, including complete languages. Every promise problem in P for which there is at least one yes-input and at least one no-input is P-complete with respect to Karp reductions, so the notion is somewhat trivial in this case. (In contrast, P-completeness with respect to log-space reductions is a more meaningful notion.) Some complexity classes may only have complete promise problems for which the promise is nontrivial and no complete languages. For instance, it is straightforward to exhibit a complete promise problem for BPP, but no BPP-complete language is known, and proving

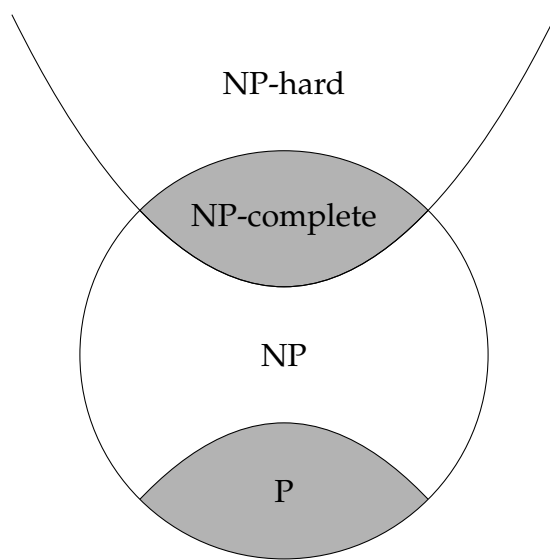


Figure 3.1: The relationship among the classes P , NP , and the NP -hard and NP -complete problems, under the assumption $P \neq NP$. The NP -complete problems sit at the intersection of NP and the NP -hard problems.

the existence of one would have major implications for our understanding of BPP . A simple example of a complexity class that provably has no complete promise problems at all is the class of all decidable promise problems — a complete promise problem for this class would contradict the time hierarchy theorem.

Perhaps the most interesting and broadly relevant notion of completeness is NP -completeness. There are thousands of known NP -complete problems, many having great practical importance. The existence of a polynomial-time algorithm for any single NP -complete problem would give polynomial-time algorithms for all of NP . Figure 3.1 illustrates the relationship among P , NP , and the NP -hard and NP -complete problems under the widely conjectured assumption that $P \neq NP$.

Here are a few basic facts about Karp reductions, all of which follow from the definition by short arguments.

1. The relation $\leq_m^P$ is transitive: if $A \leq_m^P B$ and $B \leq_m^P C$, then $A \leq_m^P C$.
2. P is closed under Karp reductions: if $A \leq_m^P B$ and $B \in P$, then $A \in P$.
3. NP is closed under Karp reductions: if $A \leq_m^P B$ and $B \in NP$, then $A \in NP$.
4. If A is NP -hard and $A \leq_m^P B$, then B is NP -hard.
5. If A is NP -complete, $B \in NP$, and $A \leq_m^P B$, then B is NP -complete.

6. If any NP-hard problem is in P, then $P = NP$.

Item 5 is a typical workhorse for proving that a given problem is NP-complete: one first checks that the problem is in NP (which is usually easy), then constructs a reduction from a known NP-complete problem. The Cook–Levin theorem, which we discuss in Section 3.4 below, is the essential starting point, as it provides a “first” NP-complete problem, from which others can be proved NP-complete through reductions.

Log-space reductions and P-completeness

The definition of log-space reductions parallels the definition of Karp reductions, with the reduction function required to be log-space computable.

Definition 3.5. Let $A = (A_{\text{yes}}, A_{\text{no}})$ and $B = (B_{\text{yes}}, B_{\text{no}})$ be promise problems over alphabets Σ and Γ respectively. We say that A *log-space reduces* to B , written

$$A \leq_m^{\log} B, \quad (3.10)$$

if there exists a log-space computable function $f : \Sigma^* \rightarrow \Gamma^*$ such that

$$x \in A_{\text{yes}} \implies f(x) \in B_{\text{yes}}, \quad (3.11)$$

$$x \in A_{\text{no}} \implies f(x) \in B_{\text{no}}. \quad (3.12)$$

The function f is called a *log-space reduction* from A to B .

Because log-space computable functions are polynomial-time computable, every log-space reduction is a Karp reduction. Hence, several of the items on the list in the previous subsection also trivially hold for log-space reductions in place of Karp reductions. Log-space reductions are also transitive, based on the composition technique for log-space computable functions from Section 3.1:

$$A \leq_m^{\log} B \quad \text{and} \quad B \leq_m^{\log} C \implies A \leq_m^{\log} C. \quad (3.13)$$

The notion of completeness extends to log-space reductions in the natural way.

Definition 3.6. For any complexity class $\mathcal{C}$, a promise problem B is $\mathcal{C}$ -hard under log-space reductions if $A \leq_m^{\log} B$ for every $A \in \mathcal{C}$. A promise problem B is $\mathcal{C}$ -complete if $B \in \mathcal{C}$ and B is $\mathcal{C}$ -hard.

Regarding NP-completeness under log-space reductions, essentially all of the classical NP-complete problems (including CIRCUIT-SAT, 3SAT, knapsack, graph colouring, and many others) are in fact NP-complete under log-space reductions. This is a consequence of the fact that the standard textbook reductions for these problems are simple syntactic transformations that log-space DTMs can execute.

The situation for P-completeness is different. In the previous subsection we observed that P-completeness with respect to Karp reductions is essentially a trivial notion, but P-completeness with respect to log-space reductions is more meaningful. An example of a P-complete language with respect to log-space reductions is CVP, the circuit value problem, which asks whether a given Boolean circuit evaluates to 1 on a given input. P-complete problems are conjectured to be inherently sequential, meaning that they do not admit efficient highly parallel algorithms.

3.3 Boolean circuits

The Boolean circuit model is an alternative to the Turing machine model that plays a key role in computational complexity. This section briefly introduces this model, discusses the notion of circuit uniformity, and sketches a proof that polynomial-time uniform families of Boolean circuits are equivalent in computational power to polynomial-time DTMs.

The Boolean circuit model

Informally speaking, a Boolean circuit is a network of AND, OR, and NOT gates connected by wires, including designated input and output wires. A more formal definition models a Boolean circuit as a directed acyclic graph together with certain node labels indicating their role.

Definition 3.7. An n -input, m -output *Boolean circuit* C consists of a directed acyclic graph, where each node has a label as described below, together with an m -tuple of distinct nodes designated as output nodes. The node labels are elements of the set $\{X_1, \dots, X_n\} \cup \{0, 1\} \cup \{\wedge, \vee, \neg\}$ and must satisfy the following conditions:

Input nodes. For each $i \in \{1, \dots, n\}$, exactly one node is labelled X_i , and every such node has in-degree 0.

Constant nodes. Every node labelled 0 or 1 has in-degree 0.

Gates. Every node labelled $\wedge$ or $\vee$ has in-degree 2, and every node labelled $\neg$ has in-degree 1.

The function

$$C : \{0,1\}^n \rightarrow \{0,1\}^m \quad (3.14)$$

computed by a circuit is defined in the natural way. Specifically, each node is assigned a Boolean value that is consistent with its label and incoming edges, beginning with the input nodes (whose values are the input bits) and constant nodes (whose values are their labels), and then propagating through the gates by evaluating the corresponding Boolean operations. The output is the tuple of Boolean values assigned to the designated output nodes.

The *size* of a circuit C , denoted $\text{size}(C)$, is the total number of gate nodes it contains. Using a standard encoding scheme that lists each node's label together with the indices of its inputs, a circuit having N nodes in total can be encoded as a binary string $\langle C \rangle$ having length $O(N \log N)$.

Universality

Boolean circuits are *universal* in the sense that every function of the form

$$f : \{0,1\}^n \rightarrow \{0,1\}^m \quad (3.15)$$

can be computed by some Boolean circuit. When $m = 1$, this follows from the expression of f in *disjunctive normal form*. More precisely, the Boolean formula

$$\bigvee_{x \in f^{-1}(1)} \left(\bigwedge_{i: x_i=1} X_i \wedge \bigwedge_{i: x_i=0} \neg X_i \right), \quad (3.16)$$

which can be converted directly into a Boolean circuit, takes the value 1 exactly when the input is a string causing f to output 1. When $m \geq 2$, we can simply build m single-output circuits and combine them into one circuit by letting them share the same input nodes.

A simple counting argument demonstrates that some Boolean functions necessarily require circuits having size exponential in the number of inputs, even when $m = 1$. In greater detail, there are 2^{2^n} Boolean functions from n Boolean inputs to one output — which is doubly exponential in n — whereas the total number of circuits of size s is only singly exponential in s . It follows that some functions (in fact most functions) require circuits of size exponential in n , because if s is sub-exponential in n we necessarily fall short of the doubly exponential number of Boolean functions that exist. In simple terms, there are not enough sub-exponential-size Boolean circuits to account for every possible Boolean function.

Circuit families and uniformity

A given Boolean circuit takes inputs of a single, fixed length, whereas a DTM can be run on inputs of arbitrary length, making the two models incomparable without additional structure. The standard remedy is to consider *families* of circuits, one for each input length.

Definition 3.8. A *Boolean circuit family* is a sequence $\mathcal{F} = \{C_n : n \in \mathbb{N}\}$ in which C_n has n inputs and a single output. The family *decides* the language

$$L(\mathcal{F}) = \{x \in \{0,1\}^* : C_{|x|}(x) = 1\}. \quad (3.17)$$

Without further restrictions, Boolean circuit families are unreasonably powerful. In fact, *every* language $L \subseteq \{0,1\}^*$, including undecidable ones, is decided by some circuit family (e.g., by taking C_n to be a circuit for the restriction of L to length- n strings based on its disjunctive normal form). Thus, the existence of a circuit family for a language implies nothing at all about that language.

One way to recover a more meaningful notion of efficient computation based on Boolean circuits is to restrict their size, though even this allows families of Boolean circuits to decide languages that DTMs cannot. The standard way to proceed is to impose so-called *uniformity conditions*, which require families of circuits that decide languages or promise problems to be *efficiently describable* by DTMs. Not only does this limit the size of each circuit in a family, but it also ensures that each circuit description is not too difficult to obtain.

Definition 3.9. A circuit family $\{C_n : n \in \mathbb{N}\}$ is *polynomial-time uniform* if there exists a polynomial-time DTM that, on input 0^n , outputs the encoding $\langle C_n \rangle$ for every $n \in \mathbb{N}$. If we require the DTM to run in logarithmic space (rather than polynomial time), we obtain *log-space uniformity*.

Log-space uniformity implies polynomial-time uniformity, while the converse is conjectured not to be true (as it would imply $L = P$). Both notions appear in the literature, but we will adopt polynomial-time uniformity as a default.

Remark 3.10. Sometimes it is convenient to extend the notion of uniformity to allow circuit families to be indexed by tuples of natural numbers, or by arbitrary strings, as opposed to single natural numbers. For example, a circuit family

$$\{C_x : x \in \Sigma^*\} \quad (3.18)$$

is polynomial-time uniform if there exists a polynomial-time DTM that, on input x , outputs the encoding $\langle C_x \rangle$ for every $x \in \Sigma^*$. We will not make any further references to this notion for Boolean circuits, but we will make extensive use of it for quantum circuits.

Equivalence with polynomial time

Polynomial-time uniform families of Boolean circuits are equivalent in power to polynomial-time Turing machines.

One direction of this equivalence is straightforward: given a polynomial-time uniform Boolean circuit family and a binary string input, a DTM can compute the corresponding circuit, then evaluate it on the input string in polynomial time by going through the gates one at a time in a so-called *topological order* that allows each gate to be evaluated.

The other required direction of this equivalence — that polynomial-time DTMs can be simulated by polynomial-time uniform families of Boolean circuits — is more substantial and represented by the following theorem.

Theorem 3.11. *Let M be a 1-DTM with input alphabet $\{0, 1\}$ that runs in time at most $p(n)$ on all inputs of length n for p a polynomial resource bound. There exists a polynomial-time uniform family $\{C_n : n \in \mathbb{N}\}$ of Boolean circuits, in which each C_n has n inputs, one output, and size $O(p(n)^2)$, that computes the same accept/reject decision as M on inputs of length n .*

The essential idea behind the proof of this theorem is represented by Figure 3.2, which suggests a Boolean circuit for simulating a 1-DTM on a length-3 binary string for 3 steps. Each of the blocks corresponds to a part of the Boolean circuit responsible for keeping track of a single tape square at one moment in time. Specifically, the contents of each tape square, along with the possible presence of the tape head and the state of the 1-DTM if the tape head is present, are encoded by a fixed-length binary string within each of the blocks. To obtain the same information corresponding to one step later requires the circuit to perform a fixed-size computation for each square, with the input being the state of that tape square component along with its two neighbouring squares. The input bits determine the initial tape contents, with as many additional blocks as required, initialized to constant values, representing blank tape squares. The sub-circuits along the leftmost and rightmost columns receive constant inputs in place of the “missing” tape squares that lie beyond the boundaries of the simulated region.

The term *tableau* is commonly used for this general arrangement, where a grid of blocks represents tape square contents at subsequent time steps. We require a tableau with $O(p(n))$ rows (one per time step) and $O(p(n))$ columns, as the tape head moves at most one square per step, giving $O(p(n)^2)$ constant-size blocks in total.

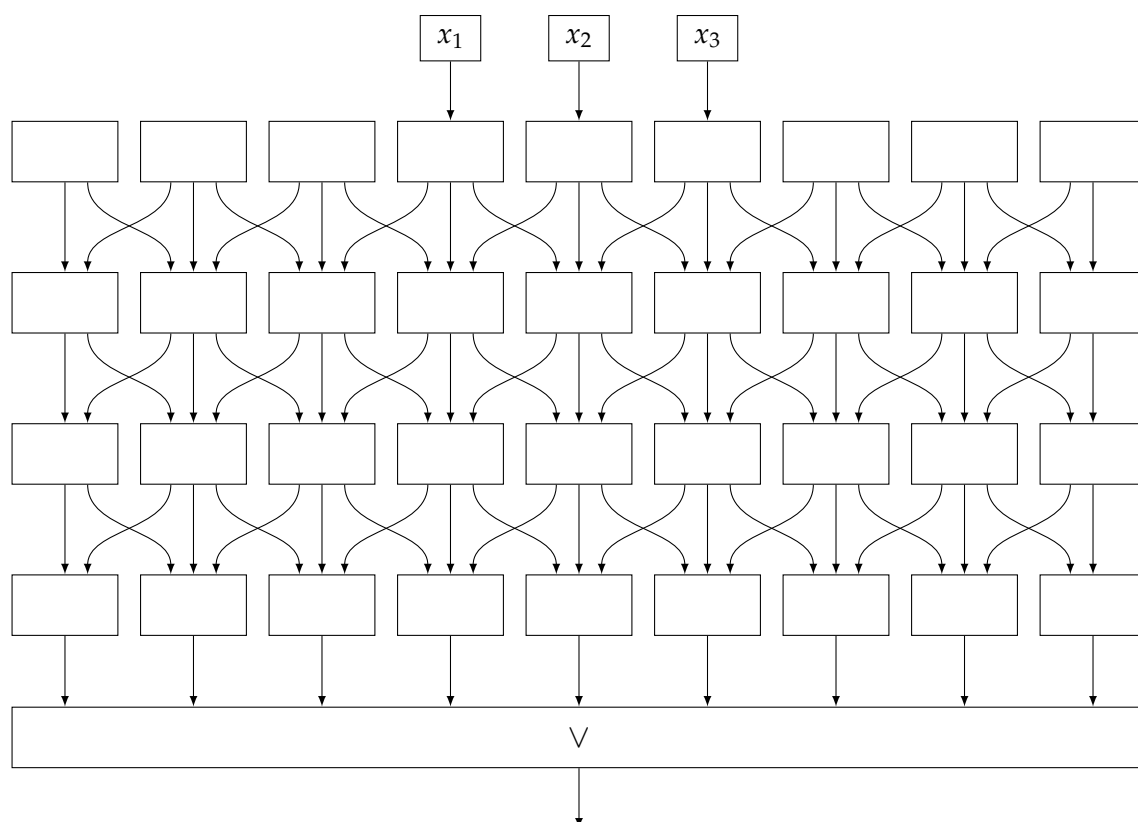


Figure 3.2: The circuit C_n obtained from the tableau of a 1-DTM M run for three steps on a 3-bit input $x = x_1x_2x_3$. Each row of blocks represents the tape at one time step; each block encodes one tape square (including its symbol and the state of the 1-DTM if the tape head is present). The content of each block depends on the three blocks above it — one for its own column and two for its closest neighbouring tape squares. We may assume that each block includes a bit indicating whether it holds the tape head in the accept state, allowing the OR gate at the bottom of the circuit to determine if the input has been accepted.

The key locality property of this circuit is that the values taken in a block in row $t + 1$ depends only on the values taken in three blocks in row t , including the one in the same column and the ones in the two neighbouring columns. Each such dependency is computed by a constant-size sub-circuit whose structure depends only on the transition function of M . Stitching together one sub-circuit for each block of the tableau produces the circuit C_n . Uniformity follows from the simple regularity of the resulting circuit, allowing a polynomial-time DTM to output its description. (Indeed, the circuit family we obtain is actually log-space uniform.)

The two directions together, along with the fact that 1-DTMs can simulate ar-

bitrary DTMs, give the following equivalence. This is a crucial tool for the Cook–Levin theorem, to be discussed in the next section.

Corollary 3.12. *A promise problem $A = (A_{\text{yes}}, A_{\text{no}})$ over the binary alphabet is in P if and only if there exists a polynomial-time uniform Boolean circuit family $\{C_n : n \in \mathbb{N}\}$ satisfying $C_{|x|}(x) = 1$ for every $x \in A_{\text{yes}}$ and $C_{|x|}(x) = 0$ for every $x \in A_{\text{no}}$.*

3.4 The Cook–Levin theorem

The Cook–Levin theorem establishes that 3SAT (defined below) is an NP-complete problem. Part of the significance of this theorem is that 3SAT serves as a kind of primordial NP-complete problem, from which thousands of others have been proved to be NP-complete through reductions. Perhaps more profoundly, it demonstrates how a collection of fixed, finite-size logical constraints on Boolean variables can precisely capture the notion of efficient verification by means of computation.

Terminology and theorem statement

Before stating the Cook–Levin theorem, let us recall some standard terminology:

- A *Boolean formula* is an expression built from variables using the operations $\wedge$, $\vee$, and $\neg$ (AND, OR, and NOT, respectively). In more detail, a Boolean variable is a formula; if ϕ is a Boolean formula then $(\neg\phi)$ is a Boolean formula; and if ϕ and ψ are Boolean formulas, then $(\phi \vee \psi)$ and $(\phi \wedge \psi)$ are Boolean formulas.
- A *literal* is either a variable or the negation of a variable. When X is a variable, it is common to write $\bar{X}$ in place of $\neg X$ to denote the negation of X , as this makes certain formulas shorter and easier to read.
- A formula is in *conjunctive normal form* (CNF) if it is a conjunction (i.e., an AND) of *clauses*, each of which is a disjunction (i.e., an OR) of literals.
- A CNF formula is a *3CNF formula* if each clause has at most three literals.
- A Boolean formula ϕ in the variables $X_1, \dots, X_n$ is *satisfiable* if there exists a Boolean assignment for the variables that causes the formula to evaluate to 1.

You can think of a clause in a CNF formula as being similar to a clause in a contract or legal agreement. That is, because the Boolean formula ϕ is an AND of a

collection of clauses, it must be that all of the clauses are true in order for ϕ to be true — just like all of the clauses in a legal document represent things that must be obeyed.

Here is an example of a CNF formula in the variables X_1, X_2, X_3, X_4 :

$$\phi = (X_1 \vee X_2 \vee X_3 \vee \overline{X_4}) \wedge (\overline{X_2} \vee \overline{X_3}) \wedge (X_1 \vee X_3 \vee X_4) \wedge (\overline{X_1} \vee X_3). \quad (3.19)$$

There are four clauses in this formula:

$$\begin{aligned} \psi_1 &= (X_1 \vee X_2 \vee X_3 \vee \overline{X_4}), \\ \psi_2 &= (\overline{X_2} \vee \overline{X_3}), \\ \psi_3 &= (X_1 \vee X_3 \vee X_4), \\ \psi_4 &= (\overline{X_1} \vee X_3). \end{aligned} \quad (3.20)$$

The formula (3.19) is not a 3CNF formula, because the first clause has too many literals, but the following formula is:

$$\psi = (X_2 \vee X_3 \vee \overline{X_4}) \wedge (X_1 \vee \overline{X_2} \vee \overline{X_3}) \wedge (X_3 \vee X_4) \wedge (\overline{X_1} \vee X_2). \quad (3.21)$$

Sometimes 3CNF formulas are defined more strictly, requiring *exactly* three literals per clause, but we will adopt the less restrictive definition for convenience.

Definition 3.13. The language 3SAT consists of encodings $\langle \phi \rangle$ of 3CNF formulas ϕ that are satisfiable.

Theorem 3.14 (Cook–Levin). 3SAT is NP-complete.

The inclusion $3\text{SAT} \in \text{NP}$ is straightforward: a polynomial-length witness can describe a satisfying assignment when there is one, which is easily checked in polynomial time. The more difficult part of the proof establishes NP-hardness, which we will sketch in two stages. First, we introduce the language CIRCUIT-SAT and argue that it is NP-hard. This is done by reducing an arbitrary problem in NP to CIRCUIT-SAT, using the tableau construction from Section 3.3. Second, we reduce CIRCUIT-SAT to 3SAT to complete the proof.

Circuit satisfiability

The circuit satisfiability problem asks whether a given Boolean circuit has a satisfying assignment.

Definition 3.15. The language CIRCUIT-SAT consists of encodings $\langle C \rangle$ of Boolean circuits C , with any number of inputs and a single output, for which there exists some input x satisfying $C(x) = 1$.

Membership of CIRCUIT-SAT in NP is straightforward: a satisfying assignment serves as the witness, and evaluating $C(x)$ takes polynomial time. The first step in the proof of the Cook–Levin theorem tackles the more challenging problem of establishing the NP-hardness of CIRCUIT-SAT.

To this end, let A be an arbitrary promise problem in NP. Our goal is to exhibit a polynomial-time reduction from A to CIRCUIT-SAT. By the definition of NP, there exist a polynomial resource bound p and a language $B \in P$ such that

$$x \in A_{\text{yes}} \implies (\exists y \in \{0,1\}^p) [\langle x, y \rangle \in B], \quad (3.22)$$

$$x \in A_{\text{no}} \implies (\forall y \in \{0,1\}^p) [\langle x, y \rangle \notin B]. \quad (3.23)$$

Given that $B \in P$, the equivalence established in Section 3.3 implies that, for each $m \in \mathbb{N}$, there exists a Boolean circuit C_m of size polynomial in m that decides B on inputs of length m , and moreover $\langle C_m \rangle$ can be computed from 0^m in polynomial time. For a fixed x of length n , the relevant value of m is $|\langle x, y \rangle|$ for $y \in \{0,1\}^p$.

From the circuit C_m we construct a circuit D_x that effectively hard-codes x into C_m , so that it takes $y \in \{0,1\}^p$ as input, forms the encoding $\langle x, y \rangle$, and feeds the result into C_m . The resulting circuit D_x satisfies $D_x(y) = 1$ if and only if $\langle x, y \rangle \in B$, and computing $\langle D_x \rangle$ from x takes polynomial time.

Now, if $x \in A_{\text{yes}}$, then there exists a string y for which $\langle x, y \rangle \in B$, so D_x is satisfiable and therefore $\langle D_x \rangle \in \text{CIRCUIT-SAT}$. If $x \in A_{\text{no}}$, then there is no such y , so D_x is unsatisfiable and therefore $\langle D_x \rangle \notin \text{CIRCUIT-SAT}$. Nothing is required for strings outside of $A_{\text{yes}} \cup A_{\text{no}}$, in accordance with the definition of a polynomial-time reduction between promise problems. The mapping $x \mapsto \langle D_x \rangle$ is therefore a polynomial-time reduction from A to CIRCUIT-SAT, and because A was an arbitrary promise problem in NP, we conclude that CIRCUIT-SAT is NP-hard.

Satisfiability of 3CNF formulas

To complete the proof of the Cook–Levin theorem, we exhibit a Karp reduction from CIRCUIT-SAT to 3SAT.

Given a Boolean circuit C with n inputs, a 3CNF formula ϕ_C is constructed over a set of variables that includes one variable for each node of C . Specifically, we have input variables $X_1, \dots, X_n$ for the input nodes and additional variables for the other nodes.

For each gate node with output variable Z and input variables W_1, W_2 (or just W_1 for a NOT gate), we include a small constant-size collection of clauses that force

Z to equal the correct Boolean function of W_1 and W_2 . For instance, the AND gate constraint $Z = W_1 \wedge W_2$ is captured by the clauses

$$(W_1 \vee \overline{Z}) \wedge (W_2 \vee \overline{Z}) \wedge (\overline{W_1} \vee \overline{W_2} \vee Z), \quad (3.24)$$

which are simultaneously satisfied precisely when Z is assigned the value $W_1 \wedge W_2$. Analogous clauses handle OR and NOT gates. For each constant node in our circuit, we include a single-literal clause that effectively fixes the corresponding variable. Finally, we include the single-literal clause requiring the variable associated with the output node to equal 1.

The total number of clauses is linear in the number of nodes of C , and the entire construction can be performed in polynomial time. The resulting formula ϕ_C is satisfiable if and only if there is a consistent assignment to the node variables that respects all gate constraints and makes the output equal to 1 — which is equivalent to some input assignment satisfying C . Thus, $\langle C \rangle \in \text{CIRCUIT-SAT}$ if and only if $\langle \phi_C \rangle \in 3\text{SAT}$.

As we already know CIRCUIT-SAT is NP-complete and $3\text{SAT} \in \text{NP}$, the existence of a Karp reduction from CIRCUIT-SAT to 3SAT implies that 3SAT is NP-complete.